



南開大學  
Nankai University

计算机学院  
软件工程实验报告

第二次作业—需求分析

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2026 年 7 月 14 日

## 摘要

智能城市生成系统面向城市场景建模、方案表达与动态分析等工作，试图把资产管理、模板驱动生成、空间布局调整、环境控制、动态仿真以及大语言模型驱动的自然语言交互整合在同一平台之中。随着人工智能与三维建模技术不断融合，传统依靠手工重复搭建的方式已经难以支撑高频迭代、多方案比较和复杂场景快速生成的需求，因此，建立一套能够支撑复杂城市场景快速生成、灵活调整与持续迭代的系统，具有较强的现实价值。

本文围绕智能城市生成系统展开需求分析。全文首先从编写目的、项目背景、任务目标、用户特点以及假定与约束出发，说明系统建设的基本动因与问题范围；随后从业务流程角度梳理项目初始化、资源准备、模板驱动生成、布局与生态控制、智能交互、动态仿真以及主系统与 Blender 插件协同之间的关系，并进一步分析系统中的核心数据对象、数据流转方式、数据字典以及版本管理要求。在功能需求部分，本文重点定义了多角色登录与权限分配、主界面与统一工作台、二维与三维资产加载及切换、模板选择与参数联动、点线面布局编辑、草图识别与布局修改、生态元素生成与环境控制、基于大模型的自然语言控制、多模态指令解析与插件自动调用、动态对象仿真、结果预览导出与版本沉淀、日志监控与后台管理等能力。同时，结合前期对 iCity 与 The City Generator 两类城市生成插件的调研，进一步讨论了本系统插件在结构控制、参数组织、交互效率和结果回写方面的设计取舍。文中还从准确性、及时性、可靠性、可扩充性、易用性、易维护性与可观测性、标准性与先进性、安全性与兼容性等角度提出了非功能需求，并分析了系统在硬件、软件、网络部署、运行维护、备份恢复与容错方面的运行要求。

整篇报告以需求分析为中心，不把重点放在底层实现细节，而是强调系统应当完成什么、这些能力为何必要、不同角色如何使用、各部分之间如何协同，以及哪些质量要求会直接影响系统能否真正落地。通过对智能城市生成系统的梳理，可以形成较为完整的需求基线，为后续概要设计、数据库设计、接口设计、原型设计以及测试验证提供清晰参考。

**关键词：**智能城市生成系统；需求分析；Blender 插件；大语言模型；三维场景建模；动态仿真；  
软件工程

# 目 录

|          |                  |           |
|----------|------------------|-----------|
| <b>1</b> | <b>引言</b>        | <b>4</b>  |
| 1.1      | 编写目的             | 4         |
| 1.2      | 项目背景             | 4         |
| <b>2</b> | <b>任务概述</b>      | <b>5</b>  |
| 2.1      | 任务目标             | 5         |
| 2.2      | 用户特点             | 5         |
| 2.3      | 假定与约束            | 6         |
| 2.4      | 可行性讨论            | 7         |
| <b>3</b> | <b>业务描述</b>      | <b>8</b>  |
| 3.1      | 总体业务场景描述         | 8         |
| 3.2      | 总体业务流程分析         | 8         |
| 3.3      | 资源准备子流程及其描述      | 10        |
| 3.4      | 模板驱动生成子流程及其描述    | 11        |
| 3.5      | 布局与生态控制子流程及其描述   | 12        |
| 3.6      | 智能交互与动态仿真子流程及其描述 | 13        |
| 3.7      | 主系统与插件协同子流程及其描述  | 14        |
| <b>4</b> | <b>数据需求</b>      | <b>15</b> |
| 4.1      | 数据需求描述           | 15        |
| 4.1.1    | 核心数据对象分析         | 15        |
| 4.1.2    | 数据质量与版本管理要求      | 16        |
| 4.2      | 数据流图             | 16        |
| 4.3      | 数据字典             | 18        |
| <b>5</b> | <b>功能需求</b>      | <b>19</b> |
| 5.1      | 功能划分             | 19        |
| 5.2      | 多角色登录与权限分配功能     | 20        |
| 5.3      | 主界面与统一工作台功能      | 20        |
| 5.4      | 插件设计参考与功能取舍      | 20        |
| 5.5      | 二维与三维资产加载及切换功能   | 21        |
| 5.6      | 模板选择与参数联动功能      | 21        |
| 5.7      | 点线面布局编辑功能        | 22        |
| 5.8      | 草图识别与布局修改功能      | 22        |
| 5.9      | 生态元素生成与环境控制功能    | 22        |
| 5.10     | 基于大模型的自然语言控制功能   | 23        |
| 5.11     | 多模态指令解析与插件自动调用功能 | 23        |
| 5.12     | 动态对象仿真功能         | 24        |
| 5.13     | 结果预览、导出与版本沉淀功能   | 24        |
| 5.14     | 日志、监控与后台管理功能     | 25        |
| 5.15     | 需求优先级划分          | 25        |

|                            |           |
|----------------------------|-----------|
| <b>6 非功能需求</b>             | <b>26</b> |
| 6.1 非功能需求的重要性              | 26        |
| 6.2 准确性需求                  | 26        |
| 6.3 及时性与性能需求               | 26        |
| 6.4 可靠性与可恢复性需求             | 26        |
| 6.5 可扩充性需求                 | 27        |
| 6.6 易用性需求                  | 27        |
| 6.7 易维护性与可观测性需求            | 27        |
| 6.8 标准性与先进性需求              | 27        |
| 6.9 安全性与兼容性需求              | 28        |
| <b>7 系统运行要求</b>            | <b>30</b> |
| 7.1 硬件配置要求                 | 30        |
| 7.2 软件环境要求                 | 30        |
| 7.3 网络与部署要求                | 30        |
| 7.4 运行维护要求                 | 30        |
| 7.5 备份、恢复与容错要求             | 31        |
| <b>8 典型使用场景与验收重点</b>       | <b>31</b> |
| 8.1 典型场景一：建模师快速搭建滨水街区      | 31        |
| 8.2 典型场景二：行业分析师核对插件功能与输出结果 | 31        |
| 8.3 典型场景三：管理员处理模板异常与版本治理   | 31        |
| 8.4 系统验收重点                 | 32        |
| <b>9 结论</b>                | <b>32</b> |

# 1 引言

## 1.1 编写目的

本文围绕智能城市生成系统展开需求分析，目的是对系统的建设目标、适用对象、业务流程、数据组织方式以及功能与非功能要求进行系统梳理，为后续的概要设计、数据库设计、接口设计和功能实现提供清晰依据。需求分析阶段的重点不是讨论具体采用哪一种实现技术，而是先说明系统需要解决什么问题、面向哪些用户、各个模块之间如何协同，以及系统应满足哪些基本质量要求。

智能城市生成系统并不是一个单纯的三维模型编辑工具，而是一套面向城市场景生成、编辑、仿真与管理的综合平台。它既要支持场景建模师在 Blender 中完成复杂建模与精细化编辑，也要支持行业分析师围绕智慧城市相关需求梳理插件应具备的功能边界，还要支持系统管理员进行权限配置、插件审核和平台维护；既要具备模板驱动和自动化生成能力，也要保留人工修正和局部覆盖空间；既要支持统一工作台，也要能够与 Blender 插件系统以及大模型服务协同工作。因此，在需求分析阶段，有必要从整体出发，对角色、业务、数据、功能和运行条件进行统一说明，避免在后续设计和实现过程中因边界不清而反复返工。

为了保证分析内容具有针对性，本文将系统范围限定在城市场景生成与分析这一核心任务链上，重点关注项目初始化、资源准备、模板调用、布局编辑、生态控制、动态仿真、结果导出和版本管理等内容，而不将更大范围的城市中台、全域 GIS 接入和实时物联网感知融合能力作为本次分析的重点。这样的范围控制有助于使需求边界更加清楚，也便于后续围绕核心功能建立完整闭环。

## 1.2 项目背景

随着智慧城市、数字孪生、城市可视化展示和空间方案推演需求不断增加，传统依靠人工逐个搭建道路、植被、水体、设施和动态对象的方式，逐渐暴露出效率低、复用性差、修改成本高以及难以快速比较多版本方案等问题。尤其在需要频繁调整天色、天气、交通密度和景观元素时，仅靠纯手工方式往往会消耗大量时间，建模工作的重点也容易被重复劳动占据。

与此同时，人工智能技术特别是大语言模型和多模态理解技术的发展，为场景生成提供了新的交互方式。过去用户如果希望让系统“把道路两侧的树木换成更疏朗的配置，并在滨水区域增加步道和休息座椅”，通常需要将这一意图拆解为多条底层命令，再在不同工具中逐步完成。现在，大模型具备将高层意图转化为结构化操作的能力，这使得城市场景生成系统不再只是传统意义上的几何编辑器，而更接近于一个能够理解用户意图、调用规则模板和插件能力、完成场景组织与动态模拟的综合平台。

从实际应用需求看，智能城市生成系统需要解决的问题主要体现在三个方面。首先是场景生成效率问题，也就是如何让用户在较短时间内完成从资源准备到基础场景搭建的全过程，而不是反复进行大量重复性摆放和替换工作。然后是交互门槛问题，也就是如何让非重度三维建模用户也能通过模板、参数甚至自然语言完成场景控制，而不必掌握过多底层建模操作。最后是分析与验证问题，也就是如何让生成出来的城市场景不只是静态画面，而能够承载天气、天色、人流、车流等条件变化，支持方案观察、比较与输出。

当然，技术能力增加并不意味着系统就会自然变得好用。任何一个真正可用的软件系统，都需要在需求层面回答很多更基础的问题，例如系统究竟服务哪些用户、最先要解决的是建模效率问题还是分析效率问题、大模型输出如何保证可控、插件自动调用是否会影响系统稳定性、资产和模板如何统一管理、不同角色之间的操作边界如何划分等。这些问题决定了系统最终是停留在演示层面，还是能够形成一个逻辑完整、结构清晰并可持续扩展的软件平台。

## 2 任务概述

### 2.1 任务目标

智能城市生成系统的总体目标，是建立一套把空间场景搭建、规则化生成、智能交互和动态仿真结合起来的软件平台，使不同角色能够围绕同一个项目环境完成方案构建、修改和分析。对于场景建模师来说，系统应当成为提高出图和起稿效率的工具，把原本重复性强的道路、绿化、设施和生态对象搭建工作转化为模板和参数驱动的流程，从而把更多时间留给细节优化与空间表达。对于行业分析师来说，系统更重要的价值在于帮助其围绕智慧城市行业需求梳理插件能力边界、制定功能标准，并结合生成结果开展验收评审。对于管理员来说，系统应当形成清晰的治理边界，让资产、模板、用户权限、项目版本和运行日志都能够被管理和追踪。

如果把这个目标进一步拆开，可以看到系统至少承担了几类非常具体的任务：

- 第一类任务是把城市中高频重复出现的对象和空间组织规则结构化，例如道路、树木、座椅、湖泊边界、滨水步道、城市外围山丘等，这样系统才能通过模板或规则快速重建这些内容。
- 第二类任务是把用户输入方式从传统手工命令拓展到图形草图、参数面板和自然语言等多种形式，使不同背景的用户都能找到适合自己的控制方式。
- 第三类任务是把静态画面延伸为可变化、可比较的动态环境，让天气、光照、人流和车流不再只是后期附加效果，而是成为方案分析的一部分。
- 第四类任务则是把以上所有能力组织在一个统一平台内，让资源、版本、日志和插件调用保持可追踪和可治理。

因此，本系统的建设目标并不只是“让软件更聪明”，而是“让城市场景构建这件事更高效、更可控、更易解释”。智能只是手段，系统化才是核心。一个真正有用的智能城市生成系统，不会只追求某个自动化演示片段，而是会关心生成前需要哪些准备、生成后怎么修改、失败了怎么办、结果如何保存、不同人如何协作等一整条链路。

### 2.2 用户特点

本系统的第一类核心用户是**场景建模师**。这类用户通常具备一定的三维建模基础，熟悉场景结构、视觉层次和资源细节，对模型精度、布局自然性和画面表现有较高要求。他们使用系统时最关心的并不是“自动化程度有多高”，而是“自动生成之后是否仍然好改、局部是否可控、资产能否快速替换、模板联动是否不会把已经修好的局部再次冲掉”。这意味着系统设计不能把建模师视为只会点击按钮的用户，而应把他们视为对结果有强控制欲的专业使用者。系统对这类用户的价值，在于帮助其更快到达可编辑的第一版，并继续在 Blender 中完成复杂建模与精细化编辑，而不是彻底取代其专业判断。

第二类核心用户是**行业分析师**。这类用户通常并不直接参与复杂场景建模，而是围绕智慧城市相关场景开展需求调研，梳理插件应具备的功能范围、参数设置方式以及结果判定标准。他们更关注系统是否贴近真实业务，插件输出是否符合预期，不同版本之间的差异是否清楚，以及生成结果是否便于核对与验收。对于这类用户来说，系统更重要的作用在于支持需求整理、功能标准制定以及结果评估。

第三类核心用户是**系统管理员**。系统管理员负责主系统的后台运维、插件上架审核及用户权限分配，同时还承担日志监控、资源治理、版本维护和异常处理等职责。他们不一定直接参与具体建模，但平台能否稳定运行、插件能否规范接入、用户能否在正确权限边界内使用系统，很大程度上取决于管

理员提供的治理能力。管理员会重点关注插件审核流程是否清晰、权限控制是否严格、日志链路是否完整以及系统状态是否可观察。

除了上述显性角色之外，系统还会受到项目负责人、算法开发者和运维人员等相关方的影响。项目负责人更关心迭代范围和交付结果是否清晰；算法开发者更关心大模型解析结果如何结构化、插件函数如何被调用、模块边界是否清楚；运维人员更关心部署条件、运行负载、日志和故障恢复手段。虽然他们未必是系统的日常一线用户，但他们的诉求会通过需求约束反映到系统设计中。

表 1: 主要利益相关者分析表

| 利益相关者    | 核心目标            | 主要关注点                    | 影响方式               | 典型矛盾点          |
|----------|-----------------|--------------------------|--------------------|----------------|
| 场景建模师    | 快速起稿并保持精细控制     | 模型质量、布局自然性、模板可改性、资源替换效率  | 决定系统是否进入真实建模流程     | 自动化太强可能削弱细节掌控  |
| 行业分析师    | 调研行业需求并明确插件功能标准 | 功能覆盖度、结果合理性、需求匹配度、验收可执行性 | 决定系统功能边界是否贴近真实业务需求 | 需求过多可能推高实现复杂度  |
| 系统管理员    | 保障平台长期稳定运行      | 后台运维、插件审核、权限分配、日志审计、系统状态 | 决定系统是否具备长期运行与治理能力  | 灵活性要求可能增加治理复杂度 |
| 项目负责人    | 控制范围与交付节奏       | 需求优先级、迭代进度、风险与可展示成果      | 决定项目边界和资源投入方式      | 追求功能过多会压缩稳定性建设 |
| 算法与插件开发者 | 让智能能力稳定接入       | 输入输出规范、任务编排、异常处理、可扩展接口   | 决定大模型与插件能力能否落地     | 需求频繁变动会造成接口不稳定 |

从上面这张表可以看出，系统内部天然存在多种目标冲突：建模师希望自动化，但前提是局部可控；分析师希望简单，但前提是结果可信；管理员希望严格治理，但又不能让系统流程僵硬到影响使用效率。需求分析最重要的意义之一，就是把这些冲突提前暴露出来，而不是等到系统快做完时才发现各方理解完全不同。

## 2.3 假定与约束

要让智能城市生成系统形成一个可落地的需求模型，必须对若干前提条件作出假设。首先，系统依赖的基础资产库和模板库应至少具备一个可以运行的最小集合，包括道路、树木、座椅、水体、山体、部分建筑和环境贴图等资源。如果没有稳定的资源基础，任何自动生成能力都会变成无根之木。其次，系统默认具备能够访问建模插件和大模型服务的运行环境，主系统、插件系统和模型服务之间可以通过既定接口完成任务传递。再次，用户愿意在必要时提供一定程度的结构化输入，例如选择模板、填写参数、上传草图或确认解析结果，而不是把所有决策都交给系统黑箱完成。

系统同时也面临一组非常现实的约束。时间约束意味着项目不可能在首轮建设中覆盖所有复杂能力，因此需求必须做优先级分层。资源约束意味着三维图形处理、大模型调用和仿真都会消耗计算资源，因此系统需要设计异步任务、状态反馈和模块解耦机制，避免单点能力拖垮整体体验。结果不确定性约束意味着草图识别和自然语言解析都不可能百分之百正确，系统必须设计确认、回滚和留痕流程，以控制自动化带来的风险。

另一个重要约束来自系统内部各模块的专业差异。资源管理关注的是命名、分类和版本；布局编辑关注的是点线面约束和空间关系；大模型解析关注的是语义理解和任务拆解；仿真模块关注的是时

间推进和行为规则。这些模块不可能用一套单一评价标准来要求，因此需求阶段必须围绕用户任务链组织系统，而不是简单按技术堆栈拆分，否则最终很容易形成模块各自完善、用户却觉得割裂的局面。

## 2.4 可行性讨论

从技术层面看，智能城市生成系统的核心能力都不是凭空出现的。三维建模和插件机制已经有较成熟的工具基础，模板驱动生成本质上是对已有规则建模方式的工程组织，自然语言解析可以借助大语言模型完成，动态仿真也可以从简化规则开始逐步增强。因此，系统的技术可行性总体较高，**真正的难点并不在于“有没有这种技术”，而在于“如何把这些技术稳定地组织在一个统一平台中”**。

从实现节奏来看，系统适合采用渐进式路线。首轮应优先保证多角色登录、资产加载、模板联动、基础布局编辑、主系统与插件协同、结果保存和日志记录这些关键闭环能力。之后再逐步增强图像草图识别、环境参数自然语言控制、多模态指令解析和复杂动态仿真等更具创新性的能力。这样的迭代路径能够先把系统做成一个能跑通的整体，再逐步提升精细程度，这也更符合真实项目的建设规律。

从组织与管理角度看，可行性还取决于需求基线是否清晰。若需求阶段没有明确哪些功能是必须先做、哪些可以延后，后续很容易在实现过程中不断扩张范围，最终每部分都做了一点，却没有任何一条流程真正闭环。因此，本文后续会对功能进行优先级划分，并把数据、接口、日志和版本要求一并纳入，目的就是让系统不只是“功能想象丰富”，而是“建设路径清楚”。

而在前期对 Blender 城市生成插件的调研中，可以看到 iCity 与 The City Generator 分别代表了两种较典型的技术路线：前者更强调道路、街区、建筑与环境之间的系统化联动生成，后者更强调集中式参数控制、快速结果反馈以及交通和夜景等视觉效果调节。基于这一调研结论，本系统的插件设计不宜简单照搬某一现成工具，而应在结构控制能力、参数化生成能力和交互效率之间取得平衡。换言之，本系统更适合吸收 iCity 在规则组织、资产联动和模块化扩展方面的优点，同时借鉴 The City Generator 在界面集中、参数直观和快速预览方面的长处，以形成更适合智能城市生成任务的插件方案。

表 2: 系统可行性分析表

| 分析维度  | 关注内容                   | 判断 | 主要依据                   | 主要风险及应对                         |
|-------|------------------------|----|------------------------|---------------------------------|
| 技术可行性 | 主系统、模板、插件、大模型和仿真模块协同实现 | 较高 | 关键技术均已有成熟基础，可通过接口方式整合  | 模块耦合过高会影响稳定性，应采用任务编排与统一接口       |
| 进度可行性 | 在有限周期内完成核心闭环           | 中等 | 可先实现基础功能，再逐步增强智能能力     | 范围膨胀会导致延期，应明确 Must 与 Should 级需求 |
| 经济可行性 | 图形计算、模型调用、资源整理与维护成本    | 中等 | 原型阶段成本可控，资源可采用分层投入方式   | 大模型调用与高精度渲染成本上升，可采用限频与缓存机制      |
| 组织可行性 | 多角色协作、模板与资源维护、日志治理     | 中等 | 职责边界相对明确，容易形成分工        | 缺少统一规范会导致后期混乱，应提前定义命名与审核流程      |
| 运行可行性 | 系统上线后的维护、升级与故障排查能力     | 较高 | 通过统一日志、配置外部化与版本控制可支撑运维 | 若早期忽略监控与追踪，后期故障将难定位             |



## 3 业务描述

### 3.1 总体业务场景描述

智能城市生成系统服务的业务，不是一次性的模型导入或单个功能测试，而是一整条从项目准备到方案输出的工作链。一个典型的使用过程通常从用户登录系统开始，随后创建项目或打开已有项目，接着加载需要使用的二维底图、三维资产和模板资源。场景构建时，用户既可以从空白画布出发通过模板和参数快速生成结构，也可以通过点线面输入、草图导入和自然语言指令对局部区域进行引导。基础结构形成之后，系统继续支持生态环境扩展、天色天气调节、车辆和人群等动态对象注入。最终，用户通过预览、仿真、版本保存和结果导出完成一次相对完整的工作闭环。

这种业务链说明，系统不能被设计成若干彼此独立的小工具拼盘。如果资源库、模板库、场景编辑器、大模型解析模块和插件执行模块彼此之间没有稳定连接，用户就只能在多个子系统之间来回跳转，不仅效率低，还容易造成版本不一致和结果丢失。因此，业务分析阶段首先要确认的一点是：主系统必须作为统一入口存在，承担项目上下文管理、任务分发、结果回写和日志记录的职责。

此外，城市场景生成与一般办公类业务还有一个显著差异，那就是“反复修改”是常态而不是例外。用户很少在第一次生成结果后就直接结束工作，更常见的情形是看到场景后继续调整道路、树木、水体边界、环境参数、仿真密度，然后再重新观察结果。也就是说，系统的业务流程天然环状的而不是线性的。需求中如果没有把这种反复迭代考虑进去，就会出现保存不清、回滚困难、修改影响范围不明等问题。

### 3.2 总体业务流程分析

为了更清楚地描述系统在不同阶段的主要动作及其前后关系，图1 给出智能城市生成系统的总体业务流程图。这个流程图并不是为了强调某个具体算法步骤，而是为了说明系统在完整工作链中如何把用户输入、资源准备、场景生成、人工修正、动态分析和版本沉淀串联起来。

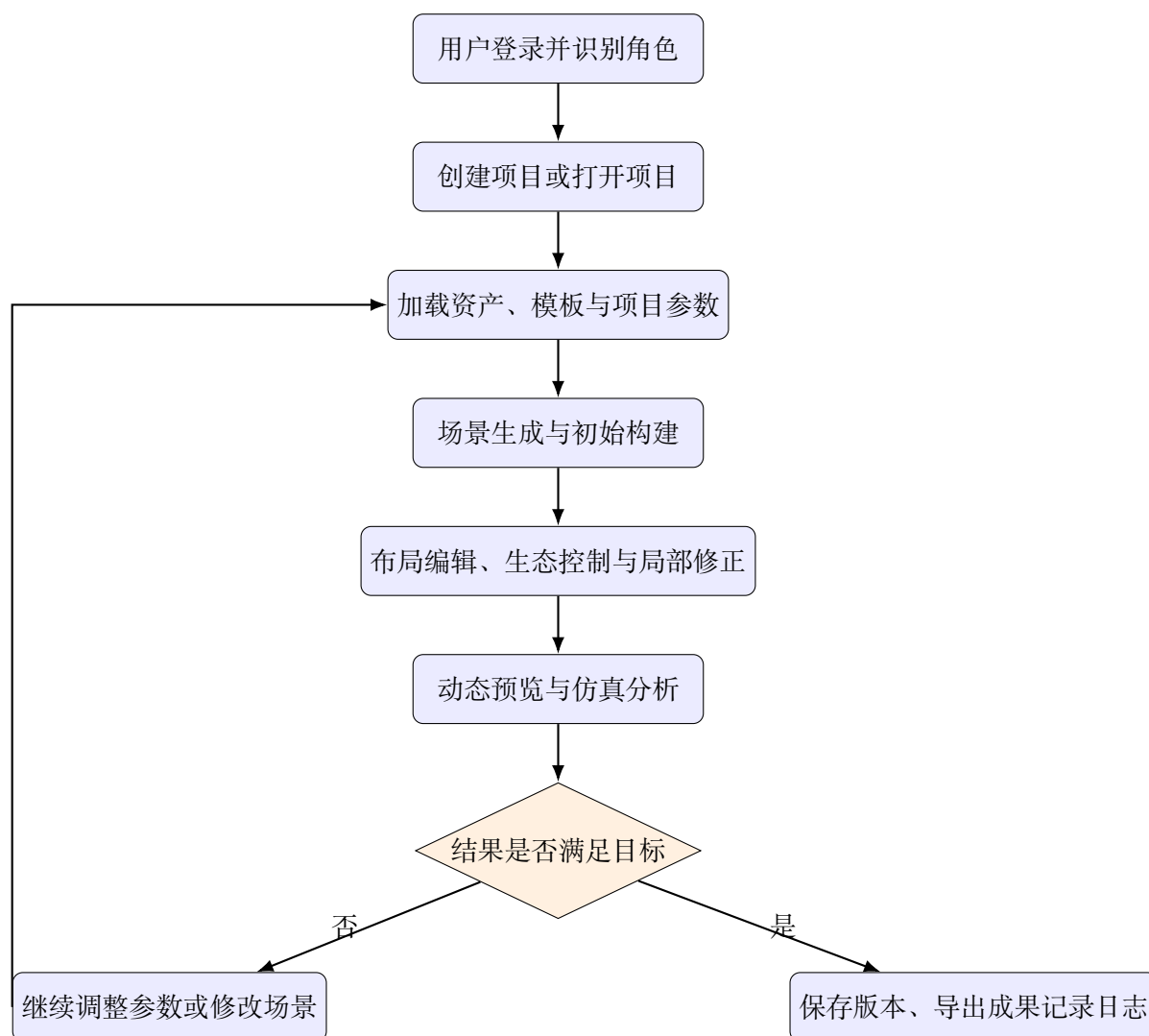


图 1: 智能城市生成系统总体业务流程

从这个总体流程来看，资源准备阶段的地位非常关键。没有经过整理的资产、模板和默认参数，后续任何自动化生成都缺乏稳定基础。许多系统在演示时容易跳过这一层，直接展示“输入一句话生成场景”的效果，但真正进入长期使用时，资源和模板的组织水平往往决定了系统输出质量的一致性。另一方面，场景生成与后续编辑之间并不是替代关系，而是连续关系。模板和大模型更适合快速生成骨架与初版结构，而布局编辑、生态调整和局部覆盖则负责让结果贴近具体需求。

业务流程中的仿真环节也不应被看成最后附加的花哨步骤。对于很多场景来说，只有经过天气、天色、人流和车流等动态条件变化之后，场景是否合理才真正显现出来。因此，动态分析不是和场景生成完全分离的独立工作，而是和前面的布局、生态和规则配置互相影响的验证环节。也正因为如此，系统需要具备版本保存和对比能力，否则用户很难回溯是哪一次修改导致了结果变化。

### 3.3 资源准备子流程及其描述

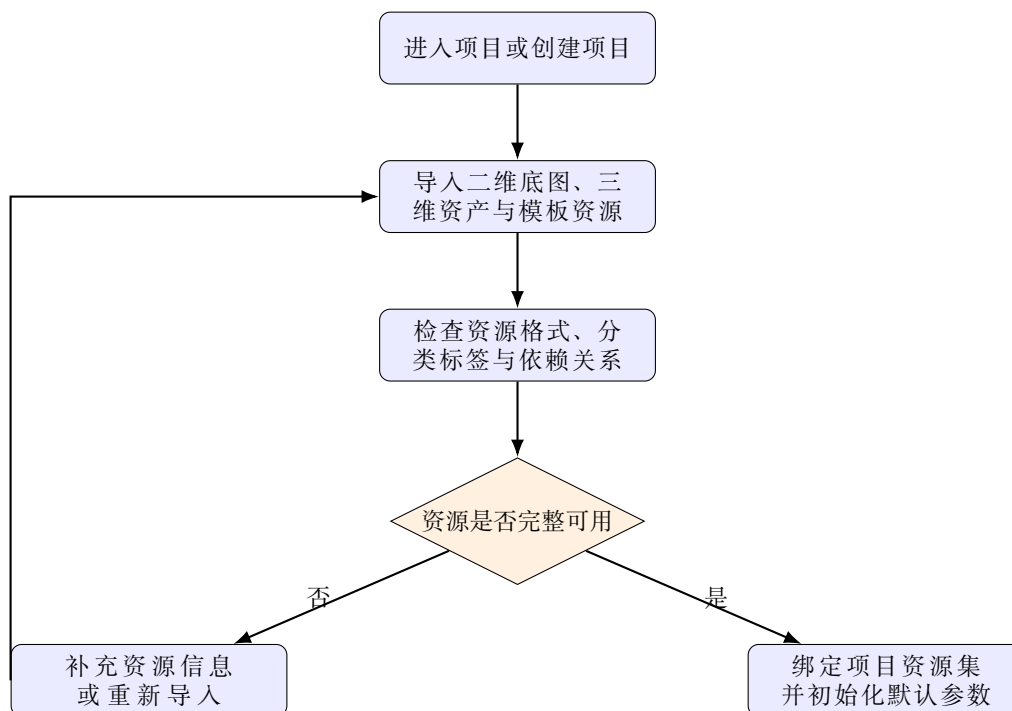


图 2: 资源准备子流程图

资源准备子流程主要围绕二维与三维资产、模板、项目参数以及外部插件所需依赖展开。用户在创建或进入项目后，首先需要确认当前项目采用哪些资源集合，哪些模板可用，以及默认参数基线是什么。这里的“资源”不仅是模型文件本身，还包括材质、贴图、分类标签、尺度说明和使用范围等元信息。没有元信息的资源即使能被系统读取，也很难被模板和智能模块稳定引用。

在这个子流程中，系统应当完成的事情至少包括资源导入、合法性校验、预览展示、分类检索、版本标识和项目绑定。比如，一类树木模型即使外观合适，如果没有尺度信息和适用场景标签，系统就无法在模板联动或自然语言解析中正确调用它。再如，某个模板可能依赖一组特定座椅模型和道路材质，如果项目中缺少这些依赖，系统就应在模板执行前给出提示，而不是等到插件运行失败后才暴露问题。

资源准备阶段还涉及默认参数初始化。道路宽度、树木密度、座椅间距、水体边缘缓冲带、环境光照基准等内容若没有初始基线，后续模板生成和仿真控制就会缺乏统一起点。需求层面应明确，项目级参数在创建时需要可配置，并且后续可以被角色允许的用户调整。只有如此，不同项目之间才能形成相对稳定的风格和规则边界。

### 3.4 模板驱动生成子流程及其描述

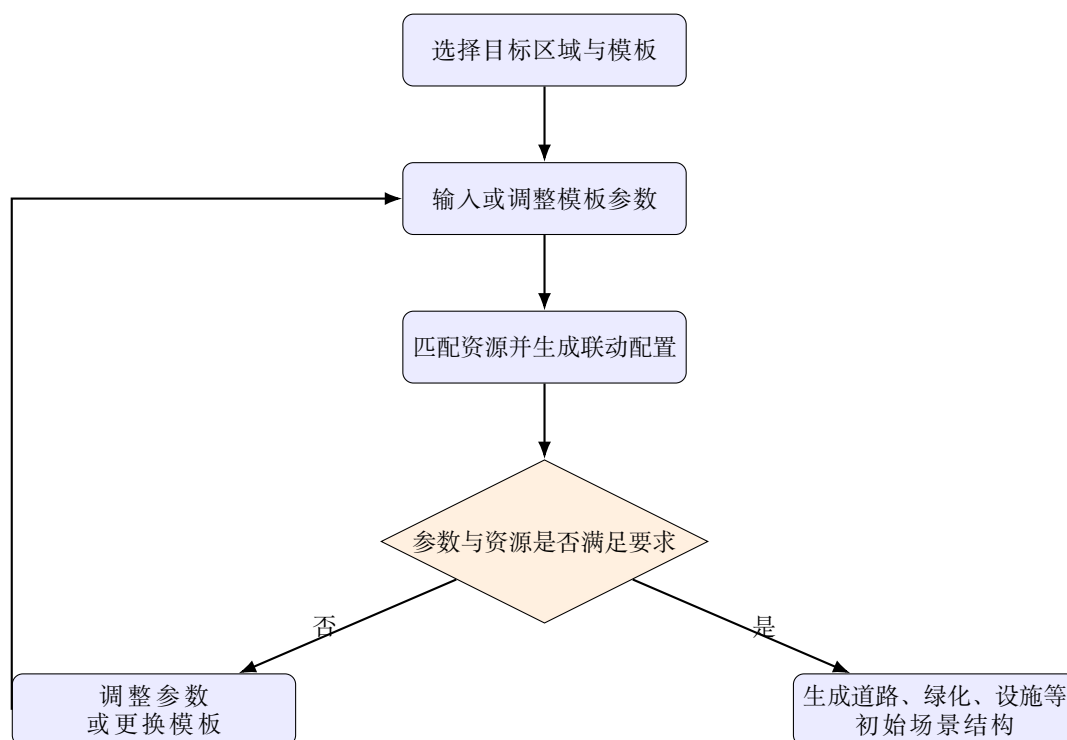


图 3: 模板驱动生成子流程图

模板驱动生成是系统提高效率的核心手段之一。模板并不是简单的预制场景，而是一组把经验规则、资源映射和参数范围封装起来的可复用建模方案。例如，某个模板可以描述“滨水步道 + 中密度绿化 + 休息座椅”的组合方式，用户只要选择该模板并输入必要参数，系统就能快速在指定区域生成一套相对合理的结构。

这一子流程的价值在于把经常重复出现的空间组织模式沉淀下来，避免用户每次都从零开始摆放。对于建模师而言，模板让他们更快到达可编辑状态；对于行业分析师而言，模板的意义更多在于把常见场景结构沉淀为清晰的功能对象，便于后续围绕插件能力进行标准制定和结果核对。然而，模板不是为了替代用户判断，因此系统还必须允许模板结果被局部覆盖、参数被手动修正、联动关系被有选择地关闭。模板若没有人工干预入口，就会从效率工具变成限制工具。

**模板生成与大模型解析之间也存在紧密关系。**自然语言交互之所以能落地，很大程度上依赖模板和规则作为“中层执行对象”。当用户输入“在湖边增加一圈适合休闲停留的步道和座椅”时，模型并不是从零发明出几何结构，而是要在已有模板、资源和规则的基础上做选择和组合。因此，模板库的语义清晰程度，会直接影响智能能力的实际效果。

### 3.5 布局与生态控制子流程及其描述

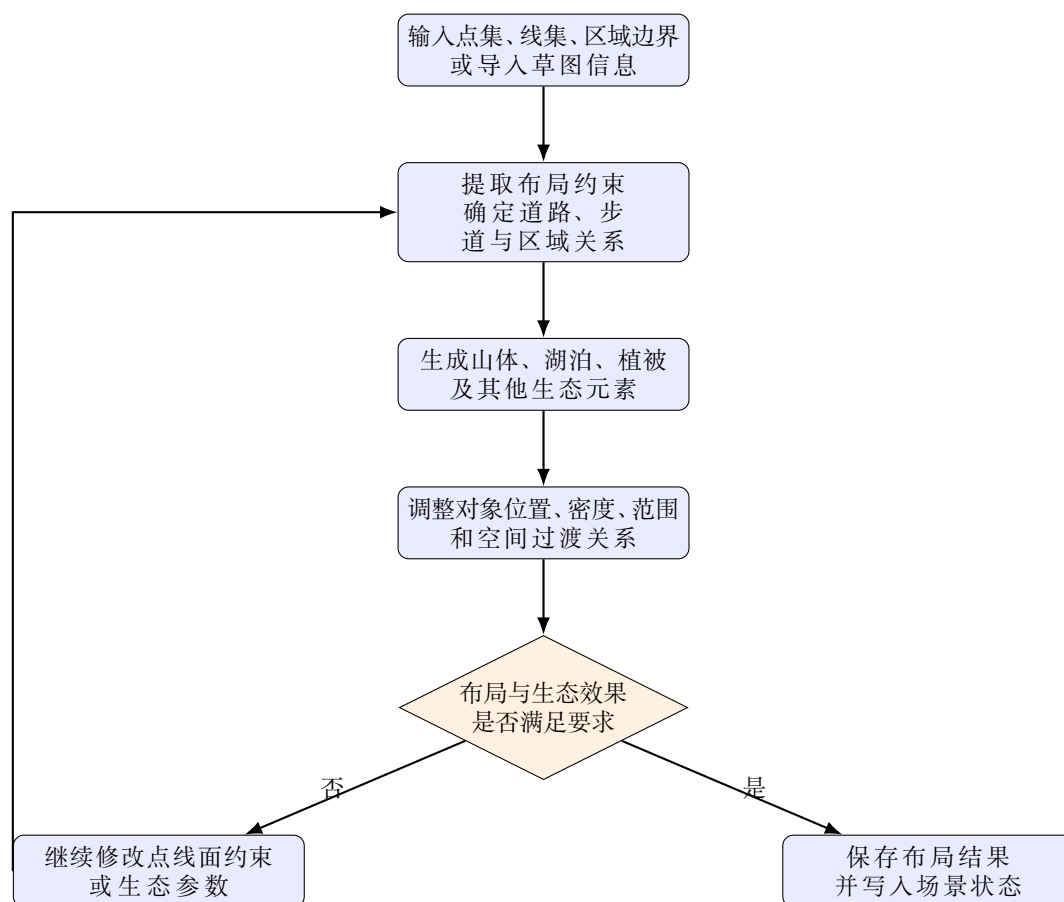


图 4: 布局与生态控制子流程图

在城市场景中，布局控制决定空间关系是否合理，生态控制决定环境表达是否完整。我们要求系统支持手动输入点集进行精确布局，也支持从图像草图中自动提取点线信息进行布局修改，同时还能在城市周边生成山丘、湖泊和动态船只等生态元素。这说明系统需要同时面向规则明确的结构化输入，以及更加直观的视觉化输入。

布局控制的核心是让用户能够以**点、线、面**形式约束道路、步道、树木、设施和区域边界的空间关系。系统应允许用户在画布中直接输入点位、绘制边线或指定区域，并将这些控制信息保存为后续可继续编辑的对象，而不是仅仅转换成一次性的模型结果。这样，当用户后续更换模板、替换资产或重新运行仿真时，系统仍然能够依据原始控制结构做更新。

生态控制则更强调对环境层次的快速组织。山体、水体、植被和船只这些对象如果完全依赖人工摆放，不仅工作量大，也很难在大场景中保持整体协调。系统应通过模板、规则或参数让这类对象能够快速生成，并允许用户根据区域边界、密度偏好和视觉效果继续微调。生态控制不只是为了让场景“更好看”，它也会影响后续仿真的空间语境和场景表达完整性。

### 3.6 智能交互与动态仿真子流程及其描述

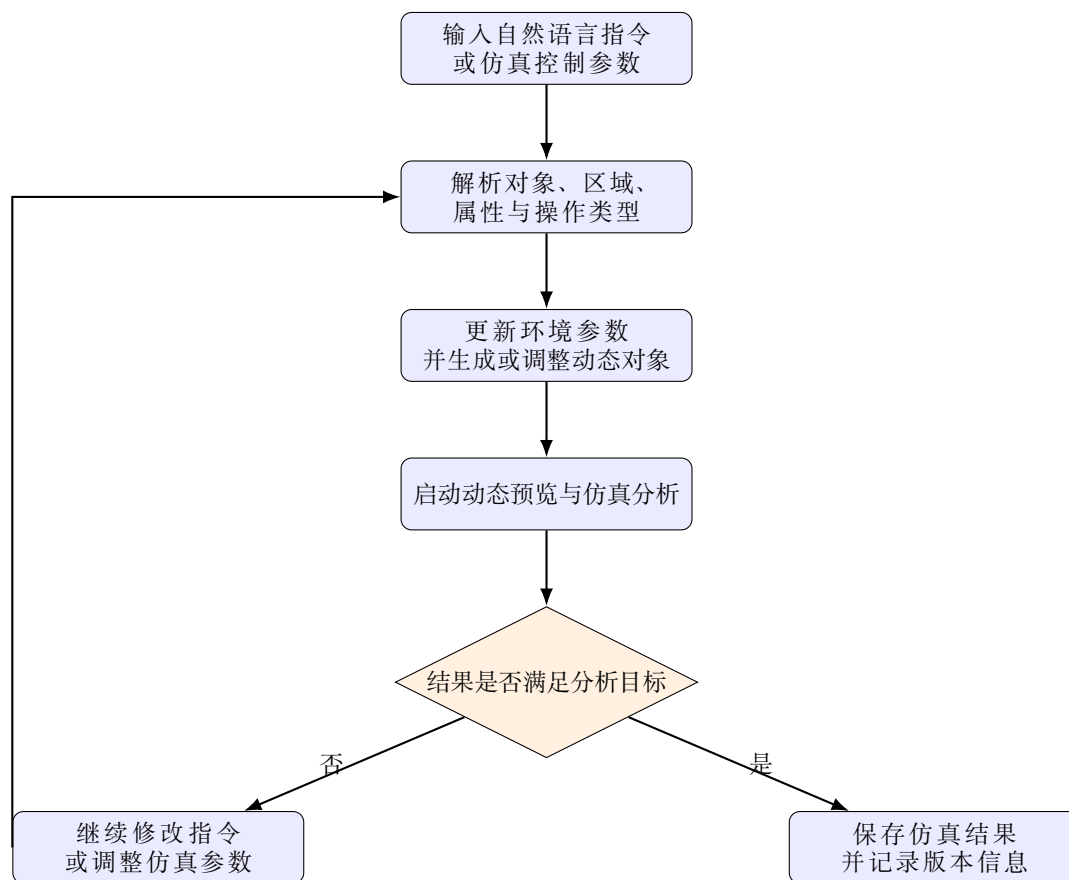


图 5: 智能交互与动态仿真子流程图

智能交互和动态仿真是本系统从“建模工具”走向“分析平台”的关键步骤。智能交互意味着用户可以用自然语言表达场景意图，例如希望把当前场景调整为傍晚、增加雨天氛围、降低机动车密度、提高步行区人流等。系统需要将这些高层描述转化为结构化操作，并结合当前项目上下文执行。动态仿真则意味着场景不再停留在静态展示上，而是能够在时间推进中呈现车辆、人群和环境变化。

这一子流程的需求重点在于解释性和可控性。大模型可以提高交互便利性，但如果系统只是简单把一句话交给模型然后直接改动场景，用户将很难知道系统究竟做了什么。因此，在自然语言解析后，系统应尽量以任务摘要或参数回显的方式告诉用户识别到了哪些对象、将执行哪些操作、作用于哪些区域。动态仿真也不应只是随机播放一段动画，而应允许用户观察仿真密度、路径约束和时间条件等参数，并支持多轮比较。

这一环节之所以重要，不仅在于它提高了交互效率，也在于系统只有把解析过程、参数变化和执行结果说明清楚，后续的结果核对、版本比较和功能验收才有依据。如果同一条指令在不同时间得到完全不一样的场景，或系统无法说清本轮仿真采用了哪些参数，分析工作就难以成立。需求分析阶段把解释性与留痕机制纳入交互链，正是为了保证系统在智能化的同时保持工程可信度。

### 3.7 主系统与插件协同子流程及其描述

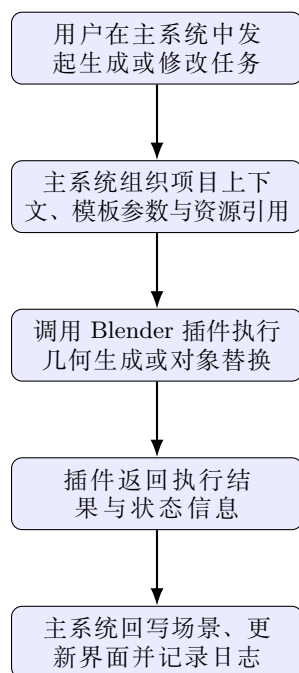


图 6: 主系统与插件协同子流程图

主系统应被视为统一工作台，它面向用户承担项目上下文、角色权限、任务管理、结果展示和日志留存等职责；插件系统则更适合承担特定几何生成、局部模型替换、场景构建函数执行等专业能力。两者之间的协作不应让用户频繁感知到边界跳转，而应尽量表现为一次完整任务在后台被自动路由和执行。

从业务角度看，用户在主系统中发起某项任务后，系统会把必要的项目上下文、选中区域、模板信息、资源引用和参数组合发送给插件。插件完成后，再将结果回写给主系统，由主系统负责更新视图、提示执行状态、保存任务记录和生成版本节点。若插件调用失败，主系统也必须负责将底层错误转换为用户可理解的提示，例如缺少依赖资源、参数冲突、运行超时或模型映射失败，而不能把原始技术报错直接暴露给终端用户。

这种主从协同关系有一个非常重要的意义，那就是把“智能”和“可治理”结合起来。大模型可以规划任务，插件可以执行任务，但真正让这套链路变成一个软件系统的，是主系统在其中承担的上下文管理、权限控制和日志沉淀工作。需求分析阶段如果没有把这层关系说明白，后续系统很容易变成若干强能力模块的简单拼接，最终难以维护。

结合已有插件调研经验来看，本系统中的 Blender 插件不应只是一个孤立的建模执行器，而应成为主系统任务链中的受控执行端。一方面，插件内部能力组织可借鉴 iCity 所体现的“城市骨架定义—建筑填充—环境细化”的分层思路，使生成结果具备较好的结构性和可调整性；另一方面，插件交互入口又应吸收 The City Generator 所体现的集中式参数面板设计，使道路、建筑、交通、夜景等高频控制项能够被快速访问和快速反馈。这样，插件既不会退化为纯黑箱工具，也不会因操作链条过长而损失使用效率。

## 4 数据需求

### 4.1 数据需求描述

智能城市生成系统虽然以“生成”“编辑”“仿真”为主要表现形式，但系统能否稳定运行，本质上取决于数据组织是否清楚。这里的数据并不是狭义的数据库表记录，而是广义上一切可以被系统识别、调用、保存和追踪的对象，包括用户与角色信息、项目与场景配置、二维与三维资产、模板定义、点线面布局控制数据、生态参数、仿真参数、自然语言任务、插件调用记录以及版本和日志等内容。如果这些对象之间关系不明确，系统就会出现生成结果难复现、参数来源难解释、插件调用难排查、版本变化难比较等问题。

从需求角度看，本系统的数据至少需要满足四方面要求：

- 首先，系统应能够完整保存项目状态，使用户在重新进入项目时，可以恢复资源引用关系、模板配置状态、布局控制信息和仿真参数。
- 其次，系统应能够支撑多轮迭代和版本对比，也就是说，除了保存最终结果之外，还需要保存导致结果变化的关键参数、任务摘要和操作记录。
- 再次，系统应能够支撑智能能力调用，这意味着模板、资源、区域对象、用户指令和插件函数之间需要存在清晰映射关系。
- 最后，系统还应具备一定的扩展能力，以便后续接入更多资源类型、模板类别和仿真对象，而不必整体重构数据结构。

#### 4.1.1 核心数据对象分析

系统中的第一类基础数据对象是**用户与权限数据**。用户对象至少应包含账号标识、角色类型、所属项目或组织范围、可访问资源范围和审计信息。权限不仅决定用户能看到哪些菜单，更决定其能否修改模板、发布资源、执行批量替换或查看高权限日志。对于一个具有多角色协作特征的平台来说，用户数据是所有业务动作的起点，也是后续日志追踪的基本前提。

第二类核心对象是**项目与场景数据**。项目可以理解为围绕某个城市片区或分析目标建立的一组长期数据集合，它包含项目名称、描述、成员范围、关联资源和默认参数等信息。场景则是项目在某个版本或方案下的具体空间状态，它包含对象集合、布局控制、生态配置、仿真配置和版本标识等内容。一个项目通常对应多个场景版本，而场景又可能在不同时间点被多次修改和保存，因此项目与场景之间天然存在层级关系。

第三类核心对象是**资产与模板数据**。资产数据描述树木模型、道路构件、建筑模块、座椅模型、贴图材质、二维底图等资源本身及其元信息；模板数据则描述这些资源如何按规则组合成某类典型空间结构。模板至少应包含名称、适用场景、参数项、默认值、联动逻辑、依赖资产和输出对象类型等内容，否则不论是用户手动选择模板，还是大模型自动调用模板，都很难形成稳定效果。

第四类核心对象是**布局与生态数据**。布局数据包括点集、线集、区域边界和空间约束，主要用于控制道路、步道、水岸线、设施分布和场地边界等几何关系；生态数据则包括山体区域、水体轮廓、植被密度和船只路径等环境表达内容。它们共同决定场景的空间秩序和环境氛围，也是后续动态仿真和局部重建的重要基础。

第五类核心对象是**任务与日志数据**。由于系统中存在大模型解析、插件调用和仿真执行等任务链，因此每一次主要操作都需要形成任务记录。任务数据应包含发起者、时间、输入内容、调用模块、执



行状态和结果摘要；日志数据则进一步记录运行轨迹、异常信息和关键中间状态。这类数据是系统可追溯性的保障，也是管理员定位问题的基础。

表 3: 核心数据对象说明表

| 数据对象  | 典型内容                 | 主要作用          | 关键要求            |
|-------|----------------------|---------------|-----------------|
| 用户与权限 | 账号、角色、访问范围、审计信息      | 确定功能边界与操作归属   | 角色清晰、越权可控、日志可追踪 |
| 项目与场景 | 项目描述、场景版本、默认参数、成员信息  | 承载完整工作上下文     | 状态可恢复、版本可比较     |
| 资产与模板 | 模型、材质、贴图、模板参数、联动规则   | 支撑场景快速生成与资源复用 | 元信息完整、引用稳定、依赖明确 |
| 布局与生态 | 点线面控制、水体边界、植被密度、山体区域 | 控制空间关系和环境表达   | 可编辑、可回放、可局部重建   |
| 任务与日志 | 指令原文、执行状态、异常信息、调用记录  | 支撑回溯、审计与故障排查  | 结构化、可查询、可关联版本   |

4.1.2 数据质量与版本管理要求

对于智能城市生成系统来说，数据质量并不仅仅是字段有没有填对，更重要的是结构是否统一、含义是否清楚、引用是否稳定、版本是否可比较。比如，同一种树木资源若在不同模板中命名混乱或缺少分类标签，系统在自动调用时就容易发生误配；若场景版本只保存最终画面而不保存参数变更和任务摘要，用户就很难解释“为什么当前版本变成了现在这样”。

因此，系统在数据质量方面至少要满足以下要求：关键对象必须具备唯一标识；资产和模板必须拥有必要元信息；模板参数应有明确的取值范围和说明；场景版本应能记录主要变更来源；日志结构应支持按项目、时间、用户和任务类型检索。只有在这些条件下，数据才能真正成为系统的稳定基础，而不是只在某个阶段勉强够用的临时载体。

版本管理在本系统中尤为重要，因为用户的典型工作方式就是反复试验和比较。需求应明确系统支持项目级与场景级版本管理，允许用户在重要节点保存快照并填写说明；支持查看版本之间的差异摘要；支持在必要时恢复到指定版本。对于由自然语言指令触发的自动操作，系统还应同时保存指令原文和执行摘要，这样后续复查时才能看清楚当前版本与某条输入之间的对应关系。

4.2 数据流图

数据流图（DFD）主要用于描述系统中的数据从何处输入、经过哪些处理过程、存入哪些数据存储，并最终输出到何处。与一般的模块关系图不同，数据流图强调的是数据在系统中的流动路径，而不是模块之间的控制调用关系。结合智能城市生成系统的需求特点，本文分别给出 0 层数据流图和 1 层数据流图，以说明系统整体边界以及系统内部主要的数据处理过程。

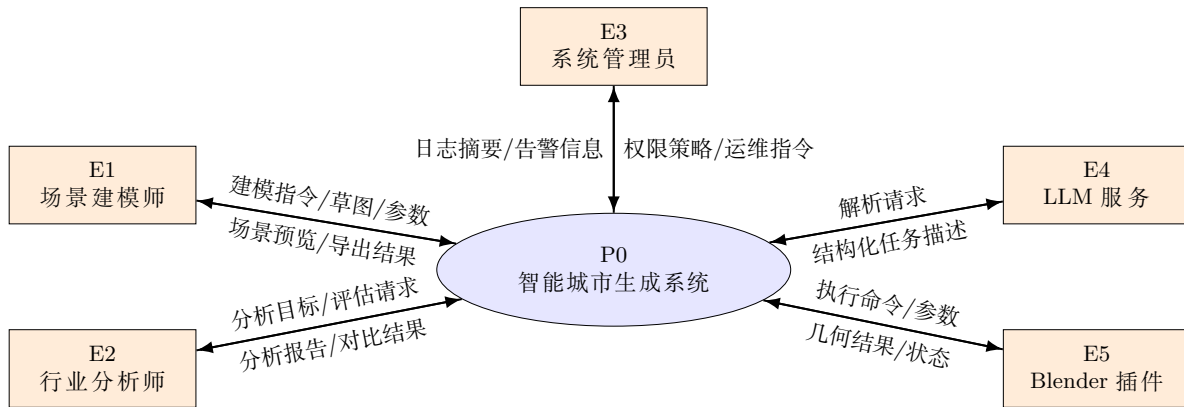


图 7: 智能城市生成系统 0 层数据流图（上下文图）

如图7所示，0 层数据流图从整体上描述了智能城市生成系统与外部实体之间的数据交换关系。系统的外部实体主要包括场景建模师、行业分析师、系统管理员、大模型服务以及 Blender 插件。场景建模师主要向系统输入建模指令、草图和参数，系统向其返回场景预览和导出结果；行业分析师向系统输入分析目标和评估请求，系统向其返回分析报告和对比结果；系统管理员则负责输入权限策略和运维指令，并接收日志摘要和告警信息。此外，系统还需要向大模型服务发送解析请求并接收结构化任务描述，向 Blender 插件发送执行命令和参数并接收几何结果与执行状态。

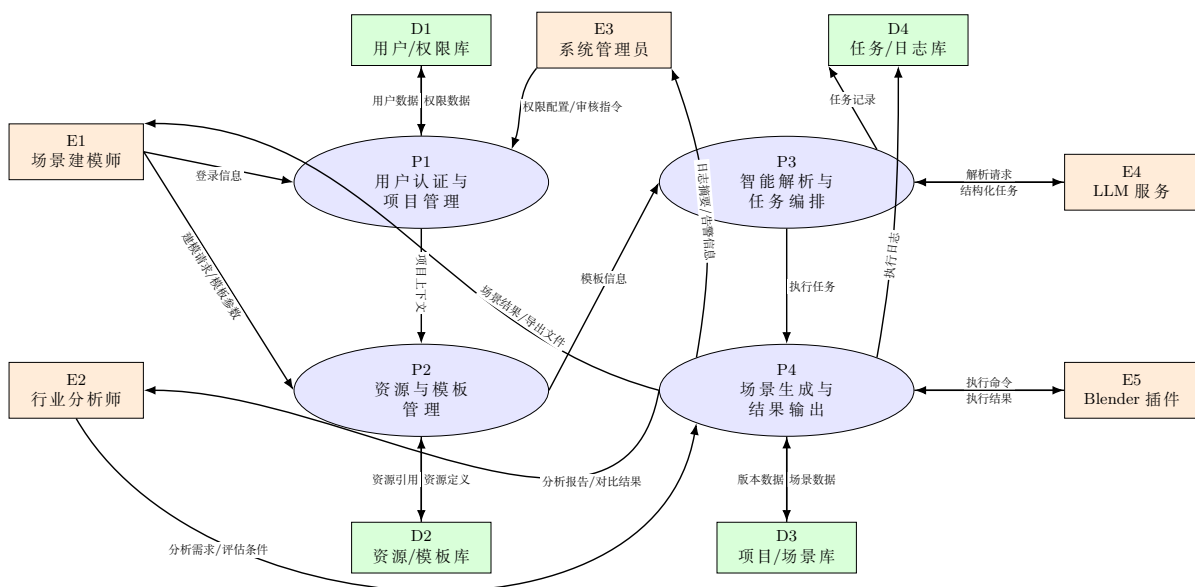


图 8: 智能城市生成系统 1 层数据流图

如图8所示，1 层数据流图进一步将系统分解为四个主要处理过程，即 P1 “用户认证与项目管理”、P2 “资源与模板管理”、P3 “智能解析与任务编排”、P4 “场景生成与结果输出”。其中，P1 主要负责用户认证、权限校验和项目上下文管理，并与 D1 “用户/权限库”进行数据交互；P2 主要负责资源导入、模板配置与参数管理，并与 D2 “资源/模板库”进行交互；P3 主要负责自然语言任务解析和执行编排，并与外部大模型服务及 D4 “任务/日志库”进行交互；P4 则负责场景生成、结果输出以及与 Blender 插件的执行协同，同时与 D3 “项目/场景库”进行场景数据和版本数据的交换。

从这两层数据流图可以看出，系统中的主要任务都应通过主系统统一进入，而不能绕过主系统直

接调用底层模块。这样做可以保证权限校验、项目上下文管理以及任务记录的一致性。同时，大模型服务的输出不应直接改写场景数据，而应先形成结构化任务，再由场景生成模块和插件执行模块完成具体处理。场景生成后的结果需要返回给用户，并写入项目/场景库形成可追踪的版本数据；同时，任务解析与执行过程中的关键信息也应记录到任务/日志库中，以支持后续的版本追踪、结果复现和异常排查。

总体而言，这两张数据流图较清楚地说明了智能城市生成系统中主要数据的输入、处理、存储与输出路径，也为后续数据库设计、接口设计和日志设计提供了基础。

### 4.3 数据字典

为了保证系统中的项目、场景、模板、资源和任务信息能够被统一管理，并为后续的功能实现、数据库设计和接口设计提供基础，下面对智能城市生成系统中的核心数据项进行定义。这里的数据字典并不等同于最终数据库表结构，而是从需求分析角度，对关键字段的名称、类型和含义进行说明，使后续设计阶段能够在统一理解下继续细化。

表 4: 系统核心数据字典

| 字段名称           | 所属对象 | 数据类型    | 可否为空 | 字段含义                        |
|----------------|------|---------|------|-----------------------------|
| user_id        | 用户信息 | varchar | 否    | 用户唯一标识, 用于区分不同系统用户。         |
| user_name      | 用户信息 | varchar | 否    | 用户姓名或显示名称。                  |
| role_type      | 用户信息 | varchar | 否    | 用户角色类型, 如场景建模师、行业分析师、系统管理员。 |
| project_id     | 项目信息 | varchar | 否    | 项目唯一编号, 用于区分不同项目。           |
| project_name   | 项目信息 | varchar | 否    | 项目名称, 用于项目展示与检索。            |
| project_desc   | 项目信息 | text    | 是    | 项目背景、目标和说明信息。               |
| scene_id       | 场景信息 | varchar | 否    | 场景唯一编号, 对应某一具体场景。           |
| scene_version  | 场景信息 | int     | 否    | 场景版本号, 用于版本控制和历史回溯。         |
| asset_id       | 资源信息 | varchar | 否    | 资源唯一标识, 对应二维或三维资产。          |
| asset_name     | 资源信息 | varchar | 否    | 资源名称, 如树木模型、道路模型、水体贴图等。     |
| asset_type     | 资源信息 | varchar | 否    | 资源类型, 如模型、贴图、材质、底图等。        |
| asset_path     | 资源信息 | varchar | 否    | 资源存储路径或资源引用地址。              |
| template_id    | 模板信息 | varchar | 否    | 模板唯一标识。                     |
| template_name  | 模板信息 | varchar | 否    | 模板名称, 如滨水休闲区模板、主干道道路模板等。    |
| template_param | 模板信息 | text    | 是    | 模板参数配置内容, 包括密度、间距、宽度等。      |
| layout_points  | 布局信息 | text    | 是    | 点集数据, 用于控制树木、座椅、道路节点等布局。    |
| layout_lines   | 布局信息 | text    | 是    | 线集数据, 用于表示道路中心线、水岸线或步道线。    |
| layout_regions | 布局信息 | text    | 是    | 区域边界信息, 用于定义功能区、水体区域或绿地区域。  |

表 5: 系统核心数据字典 (续)

| 字段名称        | 所属对象 | 数据类型     | 可否为空 | 字段含义                    |
|-------------|------|----------|------|-------------------------|
| eco_config  | 生态信息 | text     | 是    | 山丘、湖泊、植被、船只等生态元素配置参数。   |
| sim_config  | 仿真信息 | text     | 是    | 人流、车流、天气、天色等仿真参数配置。     |
| task_id     | 任务信息 | varchar  | 否    | 任务唯一编号, 对应一次生成、修改或仿真任务。 |
| task_type   | 任务信息 | varchar  | 否    | 任务类型, 如模板生成、插件执行、仿真更新等。 |
| task_status | 任务信息 | varchar  | 否    | 任务状态, 如待执行、执行中、已完成、失败。  |
| task_result | 任务信息 | text     | 是    | 任务执行结果摘要。               |
| log_id      | 日志信息 | varchar  | 否    | 日志唯一标识。                 |
| log_content | 日志信息 | text     | 否    | 日志详细内容, 用于记录关键操作与异常信息。  |
| create_time | 通用字段 | datetime | 否    | 数据创建时间。                 |
| update_time | 通用字段 | datetime | 否    | 数据最后更新时间。               |

从需求分析角度看, 我认为数据字典的意义在于让后续系统设计不再停留在模糊描述层面, 而是能够围绕这些关键字段继续细化数据库结构、接口参数和日志内容。例如, 如果系统只保存最终场景结果, 而不保存用户指令、模板参数和任务状态, 那么后续很难解释场景为何变成现在的样子; 如果资源没有统一字段描述类型和路径, 那么模板和插件执行也难以稳定工作。因此, 这份数据字典虽然还不是最终实现方案, 但已经为后续设计提供了必要的约束基础。

## 5 功能需求

### 5.1 功能划分

在智能城市生成系统中, 功能并不是彼此平行的一组按钮, 而是围绕用户任务链条逐步展开的。为了避免需求分析沦为功能堆砌, 我们在划分功能时遵循两个原则。第一, 以用户完成任务所经历的过程为主线, 而不是单纯按技术模块分割功能。第二, 以形成最小可运行闭环为目标, 即优先关注那些能够让用户真正完成一次完整建模或分析工作的能力, 而不是一开始就追求所有高级特性的全覆盖。

基于这一思路, 系统功能大体可以分为以下几个方面:

- 多角色登录与权限分配, 保证不同类型用户进入系统时拥有合适的操作空间;
- 主界面与统一工作台, 保证项目、资源、视图、任务和结果在同一入口中被组织;
- 资产加载与切换, 保证场景构建的资源基础稳定;
- 模板选择与参数联动, 保证高频结构能快速生成;
- 点线面布局控制和草图辅助修改, 保证专业用户仍然拥有较强掌控能力;
- 生态与环境控制, 保证场景表达完整;
- 自然语言与多模态智能交互, 保证用户能以更高层的方式表达意图;
- 动态仿真与结果导出, 保证场景可被观察、比较和沉淀;

- 日志、版本和管理功能，保证系统能够长期运行和维护。

## 5.2 多角色登录与权限分配功能

系统首先必须支持多角色登录，并根据角色将用户导向不同的功能视图和操作范围。角色的存在不是为了让界面看起来更完整，而是因为不同用户面对的是完全不同的任务。场景建模师更需要资源区、模板面板和对象编辑能力；行业分析师更需要查看系统输出、核对功能覆盖情况以及比对不同结果版本；系统管理员则需要用户管理、插件审核、权限配置、日志查看和系统设置。若系统不区分角色，所有功能挤在同一个界面中，既会让非专业用户无所适从，也会增加误操作风险。

**权限还需要细化到对象和动作两个层面。**对象层面决定某个用户能够访问哪些项目、资源、模板和版本；动作层面决定其能够查看、编辑、删除、发布、审核或导出哪些内容。比如，建模师可以编辑项目内场景，但未必能发布公共模板；分析师可以运行仿真和保存方案，但未必能批量替换共享资源；管理员可以查看日志并管理角色，但不一定负责具体场景搭建。这样的权限划分有助于让系统形成明确边界，也便于后续审计。

从交互上看，我觉得权限机制不应让用户频繁感受到被系统挡住，这在很大程度上影响了用户的体验。较好的方式是在用户登录后就自动加载适合的工作台，并在用户尝试越权操作时给出清晰提示，说明当前不能执行的原因。这样既能保护系统安全，也能减少其使用过程中的挫败感。

## 5.3 主界面与统一工作台功能

主界面是系统的统一入口，也是用户对整个平台的第一层认知。由于系统同时包含项目管理、资源操作、场景视图、模板配置、智能交互和任务状态等内容，主界面必须能把这些信息组织成一个逻辑清楚的工作台，而不是分散在多个相互割裂的页面中。比较合理的界面结构应包含项目区、资源区、场景视图区、参数配置区和任务反馈区，让用户在同一上下文下完成从输入到结果观察的全过程。

统一工作台的另一个意义在于承接跨模块协同。无论用户是选择模板、上传草图还是输入自然语言指令，最终都应在主界面上看到任务进入执行、状态发生变化、结果回写场景、版本被记录等过程。这样，主系统就不仅是一个可视化外壳，而是真正承担了任务组织和信息呈现的职责。尤其当底层涉及插件调用和大模型解析时，统一工作台可以成为“把技术复杂性转译为用户可理解反馈”的地方。

除此之外，界面还应体现角色差异，但这种差异不必通过完全不同的系统形态来实现。更自然的方式是在统一骨架下调整各功能区的可见性、优先级和默认展开状态。建模师进入后，资源与对象层级应更突出；行业分析师进入后，功能说明、结果对比、版本差异和验收记录应更突出；管理员进入后，插件审核、权限配置、治理和日志相关入口应更突出。这样既能保持系统一致性，也能降低学习成本。

## 5.4 插件设计参考与功能取舍

结合前期对 iCity 和 The City Generator 两款插件的调研，我认为本系统插件的设计不应单纯追求“一键生成城市”，而应围绕“结构可控、交互高效、结果可回写”三个目标展开。具体而言，插件在生成逻辑上应借鉴 iCity 所体现的系统化组织方式，即优先支持道路、街区、建筑、绿地和环境对象之间的联动生成，使插件不仅能输出视觉上像城市的场景，还能输出在结构关系上较为清晰、后续便于继续编辑和分析的场景结果。对于模板、预设和资产调用，也应采用相对模块化的设计，使不同区域类型、建筑风格和元素能够通过规则组合方式被复用，而不是仅依赖固定结果模板。

在交互方式上，本系统插件又应吸收 The City Generator 在快速创作方面的优点。对于用户高频使用的控制项，例如道路宽度、建筑高度范围、交通密度、夜景灯光、材质风格和随机种子等，应尽量

通过集中式参数面板统一呈现，减少用户在不同菜单之间来回切换的成本。插件应支持在参数调整后快速反馈结果，并提供适合大场景编辑的轻量模式或代理模式，以保证在复杂场景下仍然具备可接受的交互流畅度。

综合来看，本系统插件更适合采取“系统构建能力 + 快速交互能力”结合的设计路线：在底层生成逻辑上强调规则组织、模块联动和资产扩展，在表层交互上强调入口集中、反馈及时和操作直观。这样既可以避免纯快速生成工具在结构控制上的不足，也可以避免纯系统型工具在学习成本和操作效率上的问题。

表 6: 参考插件对本系统插件设计的启示

| 参考对象               | 可借鉴特点                                    | 在本系统中的落实方式                                | 说明                            |
|--------------------|------------------------------------------|-------------------------------------------|-------------------------------|
| iCity              | 强调道路、街区、建筑、环境之间的链路化生成；模块划分较清晰；支持预设、优化与导出 | 将插件设计为受主系统调度的生成执行端，支持模板联动、资产扩展、结构化生成与结果回写 | 重点借鉴其系统性与可扩展性，不直接照搬其较重的学习路径   |
| The City Generator | 侧栏面板集中；参数调节直接；交通模拟与夜景控制直观；结果反馈较快         | 为插件提供集中式参数面板，强化道路、建筑、交通、夜景等高频参数的统一控制与快速预览 | 重点借鉴其交互效率与直观性，不直接照搬其较弱的结构控制能力 |
| 本系统插件目标            | 同时具备结构控制与交互效率                            | 采用“规则生成 + 参数面板 + 结果回写 + 日志留痕”的组合式设计       | 在系统性与易用性之间取得平衡，服务智能城市生成场景     |

5.5 二维与三维资产加载及切换功能

系统必须支持二维与三维资产的成功导入、预览、分类和切换。二维资产主要包括底图、草图、辅助图层等内容；三维资产则包括建筑、树木、道路部件、座椅、灯杆、山体、船只等模型以及其附属材质与贴图。对于城市场景生成系统来说，资产不是简单的附件，而是场景组织的基本材料。模板、草图识别、自然语言解析甚至仿真对象注入，都需要以资源库为前提。

资产加载功能除了能够读入文件，还应具备合法性检查和元信息补全能力。系统应检查资源格式是否受支持、是否缺少依赖材质、是否存在明显尺度异常，并允许用户为资源补充名称、类型、适用场景和标签等信息。这样做的好处在于把很多问题前移到资源导入阶段，而不是等到模板执行失败或场景显示异常时再去追查。

资产切换功能则更直接服务于后续迭代。用户在已有场景中经常需要将一类树木或设施批量替换成另一类风格更合适的资源，或者根据分析目的在不同风格方案之间切换。系统应支持在对象级、区域级甚至模板级进行资源替换，并在替换前后保持引用关系清晰、日志记录完整。对于需要持续比较多个方案的项目来说，这一能力具有很高实用价值。

5.6 模板选择与参数联动功能

模板功能是系统提升建模效率的关键。在没有模板的情况下，用户每次都需要从零搭建道路、绿化、水岸设施和区域边界，效率提升非常有限。模板的意义在于把常见空间组织方式和对象联动规则封装起来，让用户通过选择模板、填入关键参数，就能快速得到一套可继续编辑的场景骨架。例如，滨水休闲区模板可以同时关联步道宽度、绿化密度、座椅间距和水岸缓冲区规则，使多个对象一次性协同生成。

参数联动功能则进一步提高了模板的实用性。实际场景中，一个参数变化往往会影响到多个对象。道路类型从主干道切换为步道时，相关绿化间距、设施朝向和可用材质范围都应随之变化。系统应让用户在模板面板中看到这些联动关系，并在参数变化时给出即时反馈。与此同时，系统也要允许用户在联动基础上做局部覆盖，因为再完善的模板也不可能适配所有具体场景。

此外，模板使用过程中的解释性也同样重要。系统应让用户区分哪些值来自模板默认、哪些值是联动自动计算、哪些值是可以自己手动修改。这样一来，当用户后续发现某个局部效果不理想时，可以更容易定位问题来源，也更有利于版本比较和经验沉淀。

## 5.7 点线面布局编辑功能

我们还要求系统支持手动输入点集进行精确布局，这意味着系统不能只是一个高层自动化生成器，还必须给用户**足够细的空间控制能力**。布局编辑功能应允许用户通过点、线、面三类基本几何约束来控制对象位置与区域结构。例如，用户可以通过输入或点击一组点位来确定树木摆放位置，通过绘制道路中心线来控制道路走向，通过划定多边形区域来定义水体、绿地或功能区边界。

对专业用户来说，这类能力决定了系统是否真正可用。因为城市场景经常存在局部细节要求，模板和自然语言交互只能解决一部分高频问题，而最终效果是否精确合理，仍然需要依赖点线面层面的细调。系统应提供吸附、对齐、间距提示、冲突检查和局部锁定等辅助功能，帮助用户更高效地完成这类操作。

更重要的是，布局编辑结果不能只停留在视觉层。点线面控制数据本身就应作为项目状态的一部分被保存下来。这样，之后模板重新计算、资产替换或仿真运行时，系统才能继续以这些原始控制信息为依据，而不是只能面对已经固化的最终模型结果。

## 5.8 草图识别与布局修改功能

草图识别功能为系统引入了一种更自然的空间输入方式。很多场景方案在最初构思时，并不是以精确坐标的形式出现，而是以简单线条、边界草图或区域示意的方式被表达出来。若系统能够从图像草图中提取点线面信息，就可以显著缩短从想法到场景结构的转化过程。用户不必一开始就精确地绘制每一条边线，而可以先用更直观的方式表达道路、水岸和空间关系，再由系统辅助完成转换。

需求上，草图识别不必追求像素级完美还原，而应聚焦于“对业务有用的结构提取”。系统需要识别主要道路线、边界线、区域轮廓和关键节点，并允许用户在正式应用前查看识别结果、手动纠正偏差、选择局部生效范围。这样做可以兼顾自动化效率和结果可控性。草图输入若没有确认和修订步骤，就很容易因为识别误差对整个场景造成连带影响。

草图功能还适合与模板结合使用。比如，用户先画出一段滨水边界和若干步道走向，系统识别之后可以自动匹配相应模板，生成沿边界展开的景观结构。这样，草图不再只是参考图，而成为真正的场景生成入口。

## 5.9 生态元素生成与环境控制功能

另外，为了进一步增强交互性，系统还应能在城市周边生成山丘、湖泊以及动态船只等生态元素，这说明场景表达不能只停留在道路和设施层面。生态元素一方面影响空间氛围和画面层次，另一方面也能为后续分析提供更完整的语境。例如，一个滨水区域如果没有水体与周边环境配合，其步道、绿化和休憩设施的布局意义就会被削弱。

生态元素生成功能应支持根据区域边界、模板规则和参数设置快速形成山体、水体、植被带和水面活动对象等内容。用户既可以通过模板直接生成一套环境配置，也可以通过输入边界和密度参数来

控制生态对象的分布范围与强弱。同时，系统应允许后续局部修正，比如改变湖泊形状、调整树木密度、增加或减少船只数量以及改变其运动路径。

环境控制还包括天色、天气、光照和氛围参数的调整。对于许多场景来说，这些要素不仅影响展示效果，也会影响分析判断。系统应支持通过界面参数和自然语言两种方式调整这些环境条件，并把每一次变化记录为可追踪的场景状态。

### 5.10 基于大模型的自然语言控制功能

大语言模型接入系统之后，最直接的价值就是让用户可以用自然语言表达高层意图，而不必手动翻译成大量底层操作。用户可以说“把主路两侧的树木调得更疏一些”“在商业街区增加夜景灯光效果”“把当前场景切换到傍晚阴天并降低车流量”，系统则负责将这些描述解析为结构化任务。

不过，自然语言控制功能的难点不在于把聊天框放进系统，而在于如何让语言真正转化为可执行、可解释、可回滚的操作。系统需要识别指令中的对象、属性、区域、操作类型和限制条件；需要判断当前项目中是否存在相应区域和资源；需要把高层意图拆解为模板选择、参数修改、插件调用或仿真设置等具体步骤；还需要在歧义较高时主动要求用户确认。只有当这些环节形成闭环，自然语言控制才是系统能力，而不是表面上的交互装饰。

因此，需求上应明确：大模型输出不应直接写入场景，而应以结构化任务描述的形式交给主系统，由主系统决定是否执行、如何执行以及怎样记录。这种做法虽然增加了一层控制，但它正是系统保持稳定和可信的关键。

### 5.11 多模态指令解析与插件自动调用功能

接入大模型实现多模态指令解析，并通过输入自然语言自动调用插件生成函数完成场景建模，这是系统最大的创新点。这一功能比普通自然语言问答更进一步，因为它要求模型不只是“理解”，还要“规划”和“调度”。用户可能同时给出文字描述、草图、当前选中的区域以及一些数值条件，系统需要把这些信息汇总成同一任务上下文，再让模型输出一套可执行的插件调用计划。

例如，用户可能先选择场景中的滨水区域，再输入“根据这段边界在外侧生成一圈慢行步道，间隔布置休闲座椅，并在靠近入口的位置加密植被”。如果系统支持多模态解析，模型就需要结合区域边界、已有资源和模板规则，规划出生成边界关联步道、再调用座椅布置函数、最后调用植被密度调整函数这样一个执行顺序。这种能力使系统真正具备从意图到操作链的自动化潜力。

但这一功能也必须建立在严格的控制基础上。模型给出的任务计划需要能被主系统审阅、能被插件接口理解、能被日志系统记录。系统应允许用户在执行前查看计划摘要，在执行后查看完成情况和异常信息，必要时可以只执行其中一部分或直接回退。也就是说，多模态解析与自动调用不应成为不可见黑箱，而应是一个被约束在工程边界内的智能流程。



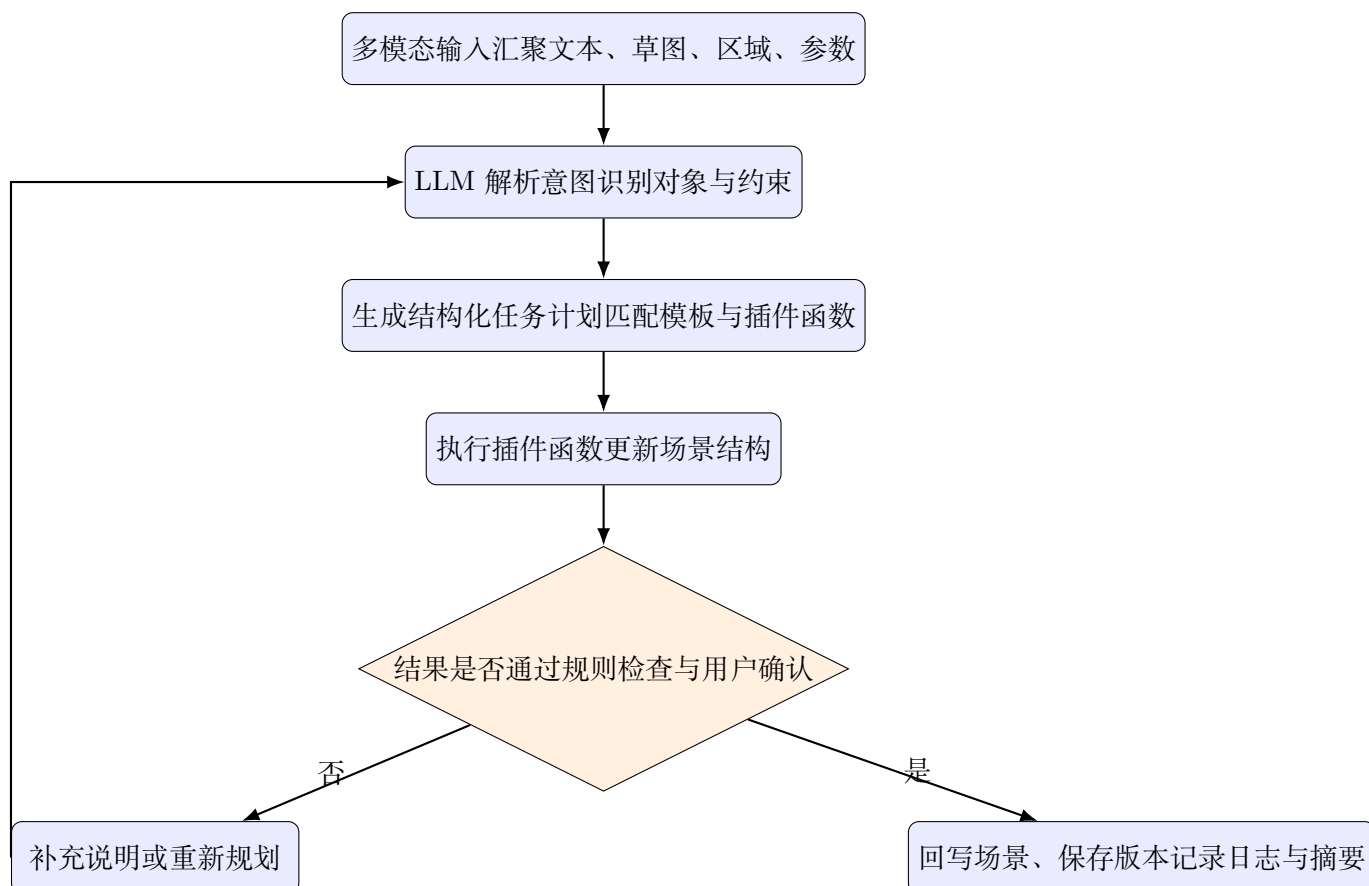


图 9: 多模态指令解析与插件自动调用流程

### 5.12 动态对象仿真功能

系统需要支持符合基本社会规则的车辆与人群动态模拟，使场景不只是一个静态画面，而能够呈现出被使用的状态。这里的仿真不必在需求阶段上升到复杂交通系统的精细建模，但至少应支持在道路和步行区域内生成动态对象、设置密度与速度区间、控制路径约束，并通过时间推进让用户观察空间使用效果。

这一功能的价值主要体现在两个方面。第一，它可以让分析师更直观地比较不同方案在使用状态下的差异，例如道路组织调整后车流是否更顺畅、滨水步道扩展后人群是否更集中。第二，它可以增强场景表达的完整性，使环境控制和空间布局不只是好看，而是能与动态活动结合起来。

需求上，仿真功能应具备基本的可配置性和可比较性。用户应能保存一组仿真参数，在不同天气和环境条件下重复运行，并把结果与其他版本进行对比。这样，仿真才不只是展示动画，而是成为方案验证的一部分。

### 5.13 结果预览、导出与版本沉淀功能

场景完成之后，系统还必须具备结果预览、导出与版本沉淀能力。预览功能需要支持多角度查看、二三维切换、关键区域聚焦、变更高亮以及参数摘要展示，让用户能够在导出前确认本轮操作到底改了什么。对于由自然语言或多模态输入触发的任务，系统尤其应把本次变化点尽量可视化，否则用户很难建立对自动化结果的信任。

导出功能应面向不同用途。建模师可能需要导出场景配置或模型结果用于进一步制作；分析师可

能需要导出截图、视频或参数报告用于比较和汇报；项目负责人可能更需要适合展示的成果文件。需求分析阶段无需穷尽所有格式，但应明确系统导出结果时需要附带必要元信息，例如版本号、时间、主要参数和来源说明。

版本沉淀功能则是把一次阶段性结果变成项目资产的关键。用户应能够在重要节点保存场景版本，填写说明，并在之后查看版本关系和摘要。对于智能自动生成的结果，原始指令和任务摘要也应被保留下来。只有如此，系统才能支撑后续复盘、比较和协作。

#### 5.14 日志、监控与后台管理功能

智能城市生成系统中存在大量跨模块调用，因此我认为日志与后台管理功能必须被当作正式需求，而不是最后才补上的维护细节。系统需要记录关键任务的发起者、输入内容、执行时间、调用模块、返回状态和异常原因；需要让管理员查看模板的使用情况、失败情况和资源依赖关系；需要对资源库、模板库、用户与角色进行统一管理；还需要在运行层面关注任务队列、失败率和服务状态。

这类功能看似不直接产生场景，但对平台稳定性至关重要。没有日志，用户无法知道某次结果为什么异常；没有模板治理，公共模板很快会变得混乱；没有任务状态监控，系统出现性能问题时也难以及时发现。

#### 5.15 需求优先级划分

由于系统功能较多，不可能在同一轮迭代中完全实现，因此有必要根据闭环价值和依赖关系进行优先级划分。表7 使用 MoSCoW 方法对主要功能进行归类。其中 Must 级需求用于构成最小可运行平台，Should 级需求用于明显提升使用体验和分析能力，Could 级需求代表创新扩展方向，而 Won't 级则表示当前阶段暂不纳入首轮目标。

表 7: 主要需求优先级划分表

| 编号  | 功能名称             | 优先级          | 说明                      |
|-----|------------------|--------------|-------------------------|
| F01 | 多角色登录与权限分配       | Must         | 系统入口能力，决定角色边界与治理基础。     |
| F02 | 主系统统一工作台与插件交互入口  | Must         | 没有统一入口，系统容易割裂为多个独立工具。   |
| F03 | 2D/3D 资产加载、预览与切换 | Must         | 场景生成的资源基础，影响模板与编辑能力。    |
| F04 | 模板选择与参数联动配置      | Must         | 直接决定系统是否具备高效起稿能力。       |
| F05 | 点线面布局编辑与局部修正     | Must         | 保证专业用户对结果的掌控能力。         |
| F06 | 生态元素生成与环境参数控制    | Should       | 显著增强场景完整性与分析语境。         |
| F07 | 自然语言控制基本能力       | Should       | 降低交互门槛，使分析型用户更容易上手。     |
| F08 | 动态对象仿真与结果比较      | Should       | 让系统具备观察和分析价值。           |
| F09 | 草图识别与布局修改        | Could        | 对早期方案构思帮助明显，但可在基础闭环后增强。 |
| F10 | 多模态解析与插件自动调用     | Could        | 创新价值高，但实现复杂度较高。         |
| F11 | 完整版本管理、日志审计与管理后台 | Must         | 关系系统可追踪性与长期运行能力。        |
| F12 | 高级监控告警与复杂调度机制    | Won't (当前阶段) | 当前阶段保留基础监控即可，复杂机制可后续补充。 |

通过这张表可以看出，系统的首要任务并不是把所有“看起来先进”的能力同时堆进去，而是先把平台基础闭环做扎实。只有 Must 级功能稳定之后，Should 和 Could 级功能的价值才能被真正释放出来。

## 6 非功能需求

### 6.1 非功能需求的重要性

**功能需求回答的是系统“做什么”，而非功能需求回答的是系统“做到什么程度才算真正可用”。**对于智能城市生成系统这样一个涉及三维场景、外部插件和大模型服务的复杂平台来说，非功能需求往往比单个功能点更能决定最终体验。一个看上去功能很多的系统，如果响应缓慢、结果不稳定、错误难排查或权限混乱，用户很快就会放弃使用。因此，在需求分析阶段把性能、可靠性、可扩展性、易用性和安全性等内容说清楚，是非常必要的。

### 6.2 准确性需求

准确性在本系统中至少体现为三层含义。第一，数据与对象映射要准确，例如模板选择后调用的资源应正确、草图识别出的边界应与原始意图大体一致、动态对象应生成在合理区域。第二，语义理解要尽量准确，大模型需要识别用户说的是哪个区域、想改什么对象、希望朝什么方向调整。第三，系统状态表达本身也要准确，用户在界面中看到的参数、版本号和任务摘要应与系统实际执行情况一致。

要实现这一点，系统需要引入适度的确认和校验机制。对于高影响操作，如大范围替换资源、自动重建布局或多步插件调用，系统应允许用户先看到任务摘要，再确认执行。对于歧义较大的自然语言输入，系统应主动澄清，而不是贸然执行。准确性不是靠猜得像来保证，而是靠结构化校验和交互确认共同维持。

### 6.3 及时性与性能需求

及时性直接影响用户对系统的主观体验。如果项目加载、模板调用或参数修改经常卡顿，自动化带来的效率收益就会被抵消。需求上，应尽量保证常规界面操作、资源检索和轻量参数调整有较为流畅的反馈；对复杂任务例如大模型解析、插件生成和动态仿真，则应采用异步执行和状态反馈机制，避免用户长时间等待却看不到任何进展。

**性能要求应体现分层思想，而不是所有操作都要求秒级完成。**登录、菜单切换、对象选中和参数面板更新这类操作应尽可能接近实时；模板生成和局部插件调用可以接受适度等待，但必须反馈任务状态；多模态解析和较大范围仿真则可以进入后台任务，由系统以进度、消息或结果列表的方式通知用户。对用户来说，可预期的等待比毫无反馈的停顿更能接受。

### 6.4 可靠性与可恢复性需求

本系统存在多处跨模块协同，因此可靠性要求尤为重要。可靠性并不等于系统绝不会出错，而是指当错误发生时，系统能够尽量保证核心功能不崩溃、已有场景不被破坏、用户知道问题出在哪里，并且有恢复路径。比如，当大模型服务暂时不可用时，系统仍应允许用户继续进行手工编辑和模板操作；当某次插件执行失败时，系统应提示失败原因并维持当前场景稳定，而不是把场景改到半途中断。

可恢复性需求则要求系统支持版本快照、操作回滚、任务重试和结果对比。特别是在自动化任务较长的情况下，如果没有中间状态保存，用户一次失败就可能损失大量工作成果。需求阶段即使不规

定具体存储策略，也应明确关键节点可保存、关键任务失败可重试、主要版本可恢复。

## 6.5 可扩充性需求

智能城市生成系统未来很可能要接入更多模板类型、更多生态规则、更多插件函数甚至更多仿真对象，因此可扩充性不应被看作后续优化项，而应从一开始就纳入需求。模板系统需要允许新增模板而不破坏已有机制；资源系统需要允许新增分类和标签体系；大模型调用层需要允许替换模型服务或增加解析策略；插件接口需要允许新增函数并保持调用规范一致。

从工程角度看，可扩充性越早考虑，后续增加能力的成本越低。如果一开始把模板、资源、插件和场景状态全都写死在某些固定流程里，后续再想扩展就会非常困难。因此，我们在前文多次强调统一接口、结构化输出和统一标识，本质上都是在为扩充性服务。

## 6.6 易用性需求

虽然系统面向的是具有一定专业背景的用户，但易用性依然十分重要。易用性并不意味着简化到没有层次，而是指用户能在合理学习成本内掌握主要任务路径，并在操作过程中得到充分反馈。建模师需要快速找到资源、模板和编辑入口，行业分析师需要清楚看到功能说明、结果差异和版本记录，管理员需要便于查看日志和配置。不同角色的“好用”定义不完全相同，因此系统应体现分层友好，而不是用同一种交互方式强行覆盖全部用户。

在具体表现上，系统应具备明确的状态提示、对象高亮、操作回显、参数说明和异常原因说明。对于初次使用系统的用户，还可以提供若干示例指令和模板推荐，帮助其更快进入任务状态。易用性和专业性并不矛盾，一个高层入口清晰、底层操作仍然开放的系统，往往比功能少但表达混乱的系统更容易被接受。

从前期对现有城市生成插件的调研可以看出，**过度分散的模块虽然有助于增强系统能力，但会明显抬高学习成本；而过度集中的单面板虽然上手较快，却可能掩盖生成逻辑。**因此，本系统在界面设计上应尽量让高频控制项集中呈现，同时为需要深入编辑的用户保留分层展开入口。

## 6.7 易维护性与可观测性需求

易维护性决定系统在投入使用之后能否持续演进。由于本系统同时涉及主界面、后台服务、插件、大模型调用和资源数据，若没有良好的维护基础，任何小改动都有可能引发连锁问题。需求上，应尽量保证配置外部化、模块接口文档化、错误日志结构化、关键流程可观察。这样，后续无论是模板升级、插件版本切换还是模型服务替换，都能较快定位问题。

可观测性是可维护性的组成部分。系统应允许管理员查看关键任务执行轨迹、异常原因和资源消耗情况，开发者也应能从日志中判断大模型解析失败、模板映射缺失还是插件执行超时。智能系统尤其不能只展示最后失败结果，而不保留中间信息，否则一旦问题出现就几乎无法追踪。

## 6.8 标准性与先进性需求

标准性需求主要体现在系统的数据组织方式、接口设计方式和模块协同方式上。由于智能城市生成系统同时涉及主系统、模板模块、仿真模块、插件系统和大模型服务，因此不同模块之间必须遵循相对统一的数据格式、命名规则和调用规范。比如，项目、场景、模板、资源和任务对象应具有清晰一致的标识方式；主系统向插件系统传递任务时，应尽量采用明确的参数结构和状态描述；日志记录也

应保持统一格式，便于后续检索、分析和故障定位。只有在这些基础规范明确的情况下，系统各模块之间才能稳定协同，后续扩展也才不会变得混乱。

先进性需求主要体现在系统应能够合理吸收当前人工智能与三维建模结合的发展成果。智能城市生成系统不应停留在传统静态建模工具层面，而应具备模板驱动生成、自然语言交互、多模态指令解析以及插件自动调用等能力，使场景生成过程更加高效和灵活。不过，先进性并不意味着一味追求复杂功能堆叠，而是要在**可控、可解释和可维护的前提下引入新技术**。也就是说，系统既要体现一定的技术前沿性，也要保证这些能力能够真正服务于实际场景生成与分析任务，而不是成为脱离业务流程的展示功能。

## 6.9 安全性与兼容性需求

安全性需求主要体现在身份认证、权限隔离、关键操作审计以及外部服务边界控制等方面。不同角色不得随意访问不属于自己的项目、模板和日志；对资源发布、模板下线和批量操作等高影响动作，应保留完整审计记录。若系统需要调用外部模型服务，则还应明确哪些内容可以传递给外部服务、哪些内容需要进行脱敏或限制。对于上传的草图、资源文件和自然语言指令，也应设置必要的输入校验与访问控制，避免异常输入直接影响系统运行稳定性。

兼容性需求主要体现在资源格式、软件环境和显示终端三个层面。系统应尽可能支持常见的二维与三维资源格式，并与 Blender 插件接口保持稳定兼容；在系统升级或插件更新后，应尽量避免已有项目因接口变化而无法继续使用。与此同时，主系统在常见显示分辨率和运行环境下应保持主要功能可用，保证项目浏览、模板调用、参数调整和结果预览等核心操作能够正常完成。需求分析阶段虽然不必穷尽全部环境，但应明确兼容性是系统长期可用的重要前提。

表 8: 关键非功能需求与验收要求

| 质量属性      | 需求说明                            | 验收关注点                              | 目标值/可测指标                                                                |
|-----------|---------------------------------|------------------------------------|-------------------------------------------------------------------------|
| 准确性       | 模板、资源、草图和语言指令应基本正确映射到场景对象与参数    | 典型用例下结果与用户意图基本一致, 高影响任务具有确认与回显机制   | 典型测试用例下, 主要对象识别与参数映射正确率不低于 90%; 高影响操作确认提示覆盖率 100%                       |
| 响应性能      | 常规操作反馈流畅, 复杂任务具备状态提示和进度反馈       | 登录、切换、参数调整不应明显卡顿, 复杂任务有明确执行状态      | 登录响应时间不超过 2 s; 页面或项目切换响应时间不超过 2 s; 普通参数修改反馈时间不超过 1 s; 复杂任务状态提示在 1 s 内出现 |
| 可靠性       | 模块失败时不破坏已有场景, 系统应能保持可继续使用       | 大模型或插件失败后项目仍可继续编辑, 错误原因可查          | 在单个模块调用失败情况下, 系统主流程可继续使用; 异常任务日志记录率 100%; 关键失败场景下已有项目数据不丢失              |
| 可恢复性      | 关键操作具备保存、回退和重试能力                | 主要版本可恢复, 自动任务失败后可重试或回退             | 场景版本保存成功率不低于 99%; 版本恢复操作成功率不低于 95%; 任务失败后支持重试, 关键操作支持回滚                 |
| 可扩充性      | 新增模板、资源类别或插件函数时不大改主流程           | 新能力能通过配置、注册或标准接口接入                 | 新增模板或资源类别时无需修改核心业务流程; 新增插件函数后可通过统一接口完成注册与调用                             |
| 易用性       | 不同角色能较快定位关键任务入口并理解系统反馈          | 主要任务路径清晰, 界面反馈完整, 异常提示可理解          | 首次使用者在指导文档辅助下, 完成核心任务平均时间不超过 15 min; 主要功能入口可在 3 次点击内到达                  |
| 易维护性与可观测性 | 配置、接口和日志结构清晰, 便于维护、排错和运行观察      | 日志可按项目、任务、时间查询, 关键配置外部化, 关键流程状态可观察 | 关键操作日志记录率 100%; 日志可按项目、用户、任务类型检索; 主要运行配置项外部化率 100%                      |
| 标准性与先进性   | 接口、数据和日志格式保持统一, 并合理引入智能生成与多模态能力 | 模块调用规范清晰, 智能能力可用且与业务流程一致           | 关键数据对象命名和接口格式统一; 模块间调用遵循统一任务描述结构; 量化指标可暂不设, 验收以规范一致性检查为主                |
| 安全性与兼容性   | 权限边界明确, 常见环境和资源格式可正常支持          | 越权访问受限, 主要格式与运行环境下系统稳定可用           | 未授权访问拦截率 100%; 系统支持常见二维/三维资源格式导入; 在规定运行环境下核心功能可正常使用                     |

## 7 系统运行要求

### 7.1 硬件配置要求

由于系统涉及三维场景显示、插件几何处理以及一定强度的动态仿真，因此硬件条件不能完全按普通信息管理系统看待。客户端至少需要具备能够流畅显示场景视图和处理基础图形交互的图形能力；如果部分插件执行放在本地，则客户端对内存和图形硬件的要求会更高。服务端方面，若主系统需要承担任务调度、资源存储和部分模型推理，则应根据任务规模配置足够的 CPU、内存以及必要时的 GPU 资源。

需求分析阶段不需要把具体硬件型号写死，但应明确系统存在图形处理和计算资源消耗，对性能较弱的设备应考虑降级策略。例如，预览可采用分级显示，复杂仿真可异步执行，部分高开销功能可只在满足条件的环境下启用。这样既能保证高配置环境下的完整体验，也能让普通环境至少维持基本可用。

### 7.2 软件环境要求

主系统、插件系统和模型服务共同构成了本系统的软件环境。主系统需要运行在稳定的服务器环境中，依赖数据库、资源存储和后台服务支持；插件系统需要与 Blender 版本和扩展接口保持兼容；若系统接入大模型服务，则还需要有可访问的推理环境或外部接口。需求层面需要明确的是，这几个部分不应独立演化到彼此不兼容，而应通过版本管理和接口规范维持协同。

软件环境要求还包括配置管理。系统中的服务地址、模板开关、资源路径、默认参数和日志级别等内容应尽量配置化，而不是直接写入代码。这样，无论是本地开发、实验室部署还是后续服务器部署，都可以在不修改大量代码的前提下完成环境切换。配置外部化是很多系统在早期容易忽略但后期代价很高的一件事，因此需求阶段最好就明确下来。

### 7.3 网络与部署要求

如果系统采用主系统统一调度、插件或模型服务独立运行的方式，那么网络连通性会直接影响功能可用性。需求上应保证主系统与资源存储、数据库、模型服务及插件执行服务之间通信稳定，并具备必要的异常处理机制。对耗时任务，系统不宜采用长时间阻塞式交互，而应配合任务队列、状态轮询或消息通知机制完成，以减少前端等待压力。

在部署形态上，系统应至少支持两种模式：一种是适合演示或小规模使用的单机或轻量部署模式，另一种是更适合多人协作的分模块部署模式。这样既能满足当前原型使用需求，也为后续扩展留出空间。部署时还应考虑管理接口保护、资源访问控制和日志输出位置等问题，避免系统刚上线就暴露明显风险。

### 7.4 运行维护要求

系统建成之后，运行维护能力决定它能否长期稳定工作。运维需求包括日志查看、服务状态检查、模板与资源管理、错误定位、版本升级和用户支持等方面。管理员需要能快速看到当前有哪些任务正在运行、哪些任务失败、哪些模板近期异常较多、资源库中哪些内容长期未使用或已过时。若这些信息缺失，系统运行时间越长，维护成本就越高。

此外，运行维护应尽量避免对用户当前工作造成大面积影响。例如，在模板升级或插件更新时，应告知用户哪些旧版本项目可能受到影响，是否需要重新绑定资源或重新运行检查。这样，系统维护不

至于变成对日常工作的打断，而能保持一种可预期、可管理的节奏。

## 7.5 备份、恢复与容错要求

考虑到系统中会积累项目数据、场景版本、模板定义、资源引用和日志信息，备份与恢复能力属于基本运行要求。系统应支持对关键项目状态和版本节点进行周期性保存，支持管理员在必要时恢复到某个指定时间点或某个关键版本。对于资源文件，可以根据重要程度采用不同备份策略，但至少应保证项目所依赖的核心资源能被追溯。

容错要求则体现在系统面对局部故障时的行为。例如，大模型服务不可用时，系统应仍允许用户进行手工编辑和模板生成；某个插件执行失败时，不应拖垮整个工作台；某次导出失败时，当前场景状态不应丢失。需求分析阶段明确这些要求，有助于后续架构设计采用隔离、重试和降级等策略。

# 8 典型使用场景与验收重点

## 8.1 典型场景一：建模师快速搭建滨水街区

设想一名建模师需要为某个滨水区域快速搭建一版可展示、可继续精修的场景。他进入系统后首先创建项目，加载底图以及与滨水环境相关的一组道路、树木、座椅、水体和建筑资源。随后，他在模板库中选择适合该区域的滨水休闲模板，设置道路宽度、绿化密度和座椅间距等参数，系统据此生成基础道路、步道和周边景观骨架。建模师接着发现局部步道与湖岸线距离偏近，于是通过点线面编辑功能手动调整边界，并将部分树木资源替换为更符合整体风格的模型。

这个场景很好地体现了我们的系统对专业用户的价值。它**不是试图让建模师完全放弃人工控制，而是在前半段帮他快速完成结构组织，在后半段保留足够的精修空间**。若系统只有自动生成，没有局部编辑与资源替换能力，那么专业用户很难真正把它纳入日常工作流程。相反，若系统只提供底层编辑却没有模板和联动，则效率提升也会十分有限。

## 8.2 典型场景二：行业分析师核对插件功能与输出结果

行业分析师的工作重点并不在于直接完成复杂建模，而在于围绕智慧城市业务梳理需求、明确插件应具备的功能，并据此核对系统输出是否符合预期。一个典型情形是，分析师在前期提出滨水休闲区、商业街区、交通主干道等典型场景的功能要求，明确插件至少应支持哪些生成能力、参数控制项和结果表现形式。随后，在场景建模师基于 Blender 插件完成初版场景构建后，分析师通过主系统查看生成结果、对照功能项覆盖情况、检查参数范围是否合理，并结合预览、仿真和版本记录进行判断。

在这个场景中，系统的关键价值并不在于让分析师亲自建模，而在于**为其提供一个能够承接需求、展示结果差异并支持功能核对的平台**。若系统只能生成结果，却无法说明采用了哪些模板、调用了哪些插件能力、输出是否满足预设要求，那么分析师就无法有效判断插件是否达到预期。

## 8.3 典型场景三：管理员处理模板异常与版本治理

管理员的工作场景与前两类用户不同。他在后台看到某个公共模板最近的失败率明显升高，于是进入日志界面查看任务明细，发现问题主要出现在一个资源映射缺失的分支上。系统应支持管理员从异常任务直接跳转到模板详情页，查看该模板依赖的资源、最近的修改记录和当前发布状态。管理员确认问题后，可以临时下线模板，或限制其只对部分项目可见，避免更多用户继续受到影响。



这个场景说明，平台类系统若缺少后台治理能力，再好的前台功能也难以长期稳定运行。因为随着模板和资源数量增加，没有日志、审批、发布和下线机制，系统迟早会出现混乱。我们在需求分析阶段就把管理员场景纳入主线，本质上是在为系统未来的长期使用打基础。

## 8.4 系统验收重点

除此之外，我认为系统验收不应只看每个功能按钮是否存在，更应看用户是否能够沿着完整任务链顺利完成工作。首先，需要验证不同角色登录后看到的工作台是否匹配职责范围，权限边界是否清晰。其次，需要验证资源加载、模板选择、场景生成、局部编辑、生态调整、自然语言控制、插件调用、动态仿真和结果保存是否能形成稳定闭环。再次，需要验证智能功能是否具备解释性，也就是自然语言和多模态输入在执行前后是否给出足够清楚的任务摘要和结果说明。

验收还应特别关注失败场景。比如，插件执行失败后项目是否仍能继续编辑，大模型解析歧义时系统是否要求确认，资源缺失时是否能及时提示而不是直接崩溃。这些内容往往比成功场景更能检验系统是否真正具备工程可用性。一个只在成功演示时看上去顺畅的系统，并不能说明它已经达到了可交付水平。

## 9 结论

综合以上分析我们可以看到，智能城市生成系统并不是一个单点功能明显的简单软件，而是一个将资源、模板、空间控制、插件协同、自然语言解析、动态仿真和平台治理组织到一起的复合型系统。它表面上看是在“生成城市”，实际上处理的是一整条从项目初始化到结果沉淀的工作链。如果仅仅把它看成三维编辑器，就会忽视模板和智能交互的价值；如果只把它看成 AI 应用，又会忽视资源质量、版本管理和平台治理的重要性。也正因为如此，我们在需求分析阶段就必须从整体出发，把角色、业务、数据、功能与质量要求统一组织起来。

从系统能力上说，我认为真正重要的不是某个局部功能多么新，而是这些功能能否围绕用户的任务形成闭环。场景建模师需要的是快速起稿后仍然能够继续精修的能力，行业分析师需要的是围绕真实需求判断插件是否做到位的能力，系统管理员需要的是清晰的治理和维护能力。模板化、自然语言、多模态解析和插件自动调用这些内容都很有价值，但它们只有建立在资源、版本、日志和权限这些基础结构之上，才能成为稳定的系统能力，而不只是单单停留在演示效果上。

因此，本文最终形成的需求基线可以概括为几点：系统应以主系统统一工作台为入口，以资产与模板为基础，以点线面和草图为布局控制手段，以自然语言和多模态交互降低操作门槛，以动态仿真和结果导出支撑分析与表达，同时通过版本、日志和权限机制保证整个过程可追踪、可治理、可恢复。后续无论是进入概要设计还是进行数据库和接口设计，都可以在这份需求分析的基础上继续展开。